



**COMSATS University Islamabad,
Sahiwal Campus, Pakistan**

CodeCollab

Collaborative Code Editor

A Final Year Project presented to
COMSATS University Islamabad, Sahiwal Campus

In partial fulfillment of the requirements for the degree of
Bachelor of Science in Software Engineering (2022-2026)

By

Ahmad Ali Raza CIIT/FA22-BSE-062/SWL

Faizan Mahmood CIIT/FA20-BSE-044/SWL

Supervisor: SE Mr. Shahzad Ali

Co-Supervisor: Ms. Abida Kouser

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied from any source. It is further declared that we have developed this software and the accompanying report entirely on the basis of our personal efforts made during this period under the guidance of our supervisor. If any part of this project is proved to be copied from any source or found to be a reproduction of some other work, we shall stand by the consequences. No portion of the work presented in this report has been submitted in support of any application for any other degree or qualification at this or any other university or institute of learning. The intellectual content of this project is the product of the authors' original work, and all external sources have been properly cited.

Ahmad Ali Raza

CIIT/FA22-BSE-062/SWL

Faizan Mahmood

CIIT/FA20-BSE-044/SWL

CERTIFICATE OF APPROVAL

It is hereby certified that the Final Year Project of BS (Software Engineering) titled "CodeCollab" was developed by AHMAD ALI RAZA (CIIT/FA22-BSE-062/SWL) and FAIZAN MAHMOOD (CIIT/FA20-BSE-044/SWL) under the supervision of SE MR. SHAHZAD ALI and Co-Supervisor MS. ABIDA KOUSAR, and that in their opinion it is fully adequate in scope and quality for the degree of Bachelor of Science in Software Engineering (2022-2026).

Supervisor

SE Mr. Shahzad Ali

Lecturer, Department of Software Engineering

COMSATS University Islamabad, Sahiwal Campus

Head of Department

Department of Software Engineering

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

CodeCollab is a browser-based collaborative development platform that unifies real-time code editing, multi-language execution, team communication, and professional-grade development tooling in a single accessible environment. Developed as a Final Year Project under the Agile methodology, it addresses critical gaps left by existing platforms such as CodePen, Replit, and VS Code Live Share particularly the absence of integrated team communication and collaborative debugging. The platform provides real-time collaborative editing via Operational Transformation, execution support for 10+ programming languages through isolated Docker containers, synchronized collaborative debugging, advanced code intelligence powered by the Language Server Protocol (LSP), project-wide search via Elasticsearch, and Git-based version control all accessible directly from the browser without any local setup.

Built on React.js 18.x, Node.js 20 LTS, MongoDB 7.x, Docker 24.x, Socket.io 4.x, Redis 7.x, and Elasticsearch 8.x, the system is structured around 18 functional modules covering user authentication, real-time collaboration, code execution, payment processing, and administrative management. Security is enforced through JWT-based authentication, Role-Based Access Control (RBAC), HTTPS, Helmet.js middleware, and GDPR-aligned data handling. A freemium subscription model offering Free, Premium, and Institutional tiers via Stripe and PayPal alongside a centralized Admin Panel makes CodeCollab suitable for both individual developers and large-scale institutional deployment. The project is developed by two team members: Ahmad Ali Raza (CIIT/FA22-BSE-062/SWL) as Backend and Infrastructure Lead overseeing 11 backend modules, and Faizan Mahmood (CIIT/FA20-BSE-044/SWL) as Frontend and UX Lead responsible for 7 frontend modules, collaborating through shared API contracts and a unified technology stack.

Acknowledgement

All praise is to Almighty Allah, the Most Gracious and the Most Merciful, who bestowed upon us the knowledge, ability, and determination to accomplish this challenging academic endeavor. Without His guidance and blessings, the completion of this project would not have been possible.

We are profoundly grateful to our project supervisor SE Mr. Shahzad Ali for his invaluable guidance, consistent encouragement, and constructive feedback throughout the entire development journey. His technical insights and academic mentorship have been instrumental in shaping CodeCollab into a comprehensive, well-structured platform. We are equally thankful to our Co-Supervisor Ms. Abida Kousar for her technical assistance, milestone evaluations, and domain-specific mentoring that helped us navigate the complexities of this project. We would also like to extend our heartfelt appreciation to the Department of Software Engineering at COMSATS University Islamabad, Sahiwal Campus, for providing us with the academic environment, resources, and framework necessary to undertake and complete a project of this scale. The Final Year Project Committee's structured evaluation process motivated us to maintain high standards throughout the development lifecycle.

Finally, we are deeply thankful to our parents and families for their unwavering support, patience, and encouragement throughout our academic journey. Their belief in our capabilities has been a constant source of motivation and strength.

Ahmad Ali Raza

Faizan Mahmood

Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
UI	User Interface
UX	User Experience
IDE	Integrated Development Environment
OT	Operational Transformation
LSP	Language Server Protocol
JWT	JSON Web Token
MFA	Multi-Factor Authentication
OTP	One-Time Password
DB	Database
NoSQL	Not Only SQL
SDLC	Software Development Life Cycle
MVVM	Model-View-View Model
NFR	Non-Functional Requirements
FR	Functional Requirements
CRUD	Create, Read, Update, Delete
JSON	JavaScript Object Notation
HTTPS	Hypertext Transfer Protocol Secure
CORS	Cross-Origin Resource Sharing
RBAC	Role-Based Access Control

GDPR	General Data Protection Regulation
CI/CD	Continuous Integration / Continuous Delivery
SSO	Single Sign-On
REPL	Read-Evaluate-Print Loop
REST	Representational State Transfer
SRS	Software Requirements Specification
SDD	Software Design Document
UC	Use Case
BR	Business Rule
WS	WebSocket
DAP	Debug Adapter Protocol
ORM	Object-Relational Mapping

Table of Contents

1. Introduction	2
1.1.1 Purpose of the Project	3
1.1.2 Scope of the System	3
1.1.3 Technologies Used	4
1.1.4 System Overview	5
1.1.5 Outcome of the Project	6
1.2 Relevance to Course Modules	6
1.2.1 Software Engineering	6
1.2.2 Database Systems	7
1.2.3 Web Engineering and Internet Programming	7
1.2.4 Object Oriented Programming	7
1.2.5 Data Structures and Algorithms	8
1.2.6 Human Computer Interaction	8
1.2.7 Information Security	8
1.3 Project Background	8
1.4 Literature Review	9
1.5 Analysis from Literature Review	10
1.5.1 Alignment with Real-Time Collaboration Trends	10
1.5.2 Validation of Technology Choices	10
1.5.3 Gaps Addressed by CodeCollab	10
1.5.4 Security and Code Execution Safety	11
1.5.5 Educational Technology Research	11
1.6 Methodology and Software Lifecycle	11
1.6.1 Agile Methodology	11
1.6.2 MVVM Architecture	12
1.6.3 Sprint Structure	12
1.6.4 Team Work Division	12
1.6.5 Testing and Quality Assurance	13
2. Problem Definition	15
2.1 Problem Statement	15

2.2 Deliverables and Development Requirements -----	16
2.2.1 Deliverables -----	16
2.2.2 Development Requirements -----	17
2.3 Current System Analysis -----	19
3. Requirement Analysis -----	22
3.1 Introduction to Requirement Analysis -----	22
3.2 Actors in the System -----	22
3.3 Use Case Diagrams -----	23
3.4 Use Case Descriptions -----	25
3.5 Functional Requirements -----	36
3.6 Non-Functional Requirements -----	39
4. Design and Architecture -----	44
4.1 System Architecture -----	44
4.1.1 Architecture Style -----	44
4.1.2 System Architecture Components -----	45
4.2 Data Representation -----	47
4.2.1 Database Design and Schema -----	47
4.2.2 Entity-Relationship Representation -----	49
4.3 Process Flow -----	49
4.3.1 User Authentication and Session Management -----	49
4.3.2 Real-Time Code Collaboration Workflow -----	50
4.3.3 Code Execution Workflow -----	52
4.3.4 Payment Processing Workflow -----	52
4.5 Data Design -----	54
4.5.1 Data Architecture -----	54
4.5.2 Database Design -----	55
4.5.3 Data Storage Design -----	56
4.5.4 Data Flow and Synchronization -----	56
4.5.5 Data Security and Integrity -----	57
5. Implementation -----	59
5.1 Core Algorithms -----	59
5.1.1 Operational Transformation Algorithm -----	59

5.1.2 Code Execution Security Algorithm -----	60
5.1.3 Authentication Algorithm -----	61
5.1.4 Real-Time Notification Dispatch Algorithm -----	61
5.2 Tools and Technologies -----	61
5.3 External APIs -----	64
5.4 User Interface Design -----	66
5.4.1 Landing Page and Authentication -----	66
5.4.2 Project Dashboard -----	67
5.4.3 Main Workspace Layout -----	68
5.4.4 Real-Time Collaboration Visual Indicators -----	69
5.4.5 Admin Panel Interface -----	69
6. Testing and Evaluation -----	71
6.1 Manual Testing -----	71
6.1.1 System Testing Overview -----	71
6.1.2 Unit Testing -----	71
6.1.3 Functional Testing -----	76
6.1.4 Integration Testing -----	80
6.2 Automated Testing -----	82
7. Conclusion -----	87
7.1 Future Work -----	87
7.1.1 AI-Powered Code Assistance -----	87
7.1.2 Mobile Applications -----	88
7.1.4 Enterprise Features -----	88
7.1.5 Plugin and Extension Marketplace -----	88
7.1.7 Enhanced Educational Analytics -----	88
8. References -----	89

List of Figures

Figure 1 : Use Case Diagrams -----	24
Figure 2 : Architecture Style -----	45
Figure 3 : Architecture Style -----	50
Figure 4 : Architecture Style -----	51
Figure 5 : Payment Processing Workflow -----	53
Figure 6 : Data Architecture -----	55
Figure 7 : CodeCollab Landing Page -----	67
Figure 8 : Project Dashboard -----	68
Figure 9 : Main Workplace layout -----	68
Figure 10 : Multiple User Cursors with Color-Coded Labels -----	69

List of Tables

Table 1 : Comparative Analysis of Existing Platforms and CodeCollab Solution -----	9
Table 2 : Team Members Work Division -----	12
Table 3 : CodeCollab Technical Development Requirements -----	17
Table 4 : Summary of Current System Limitations and CodeCollab Responses -----	19
Table 5 : CodeCollab System Architecture Component Summary -----	45
Table 6 : Core MongoDB Collection Schema Reference -----	47
Table 7 : Complete Tools and Technologies Used in CodeCollab -----	61
Table 8 : External APIs and Integrations Used in CodeCollab -----	64
Table 9 : CodeCollab Unit Testing Results -----	71
Table 10 : Functional Testing Results -----	76
Table 11 : Integration Testing Results -----	80
Table 12 : Automated Testing Summary -----	82
Table 13 : Non-Functional Performance Testing Results -----	84

Chapter 1

INTRODUCTION

1. Introduction

The modern software development landscape has undergone a significant transformation driven by the rapid adoption of remote work and distributed teams. According to the Stack Overflow Developer Survey 2023, over 58% of professional developers now work remotely, creating a growing demand for tools that support real-time collaboration without requiring physical co-location. Despite this demand, existing platforms each carry notable limitations that prevent them from serving as comprehensive solutions. CodePen is largely restricted to frontend technologies, Replit lacks granular role-based access control and secure containerized execution, and GitHub Codespaces or VS Code Live Share require either billing configuration or a locally installed editor making them inaccessible to users without prior environment setup. No existing platform simultaneously provides real-time collaboration, secure isolated code execution, professional editor intelligence, and organizational management within a single self-contained web application. CodeCollab is proposed as a direct response to these identified gaps. It is a fully browser-based collaborative code editor that delivers a complete, professional-grade development experience without requiring any local installation or environment configuration. Built on React.js and Node.js following a modular, scalable architecture, the platform executes code through isolated Docker containers ensuring consistent runtime environments and eliminating the common "works on my machine" problem. Editor intelligence features including syntax checking, auto-completion, inline diagnostics, and symbol navigation are implemented through the Language Server Protocol, the same open standard powering VS Code and JetBrains IDEs. Real-time collaboration is enabled via WebSocket communication, allowing multiple users to simultaneously edit files, track cursor positions, and observe changes with minimal latency supporting use cases ranging from pair programming and technical interviews to instructor-led academic sessions. The platform supports execution across multiple languages including Python, JavaScript, Java, and C++, and extends its core functionality through an Admin Panel for role and workspace management, a freemium subscription model with Stripe and PayPal integration, a real-time notification system, and a security framework aligned with OWASP Top Ten standards collectively positioning CodeCollab as a robust solution for both professional development teams and academic institutions.

1.1.1 Purpose of the Project

The fundamental purpose of CodeCollab is to engineer and deliver a comprehensive, all-in-one, browser-accessible collaborative development environment that eradicates the necessity for multiple, disconnected, and often incompatible software tools. By centralizing the entire development workflow—from initial authoring and version control to execution and deployment—into a singular, unified interface, the platform aims to significantly minimize context-switching. This reduction in cognitive overhead allows developers to maintain a state of "flow," thereby substantially augmenting team productivity and accelerating the software delivery lifecycle. Furthermore, CodeCollab seeks to democratize access to professional-grade development resources by leveraging cloud-native technologies, making high-performance computing environments available to any user possessing a device equipped with a modern web browser, regardless of local hardware specifications.

In educational contexts, the platform is purpose-built to transform pedagogical methodologies. CodeCollab empowers instructors to transition from passive lecturing to active, live coding sessions where they can monitor student code in real-time, observe logic formation as it happens, and provide granular, in-editor feedback. This capability facilitates immediate intervention and correction, which is critical for teaching complex programming concepts. In professional contexts, the platform serves as a vital bridge for geographically distributed teams, enabling them to collaborate on shared codebases with a richness of interaction that effectively mirrors physical proximity. By providing instant visibility into peer contributions and synchronized debugging environments, CodeCollab ensures that software development velocity and code quality standards are maintained despite geographic dispersion, thereby eliminating the traditional friction associated with remote engineering workflows.

1.1.2 Scope of the System

CodeCollab is architected as a robust, web-based real-time code collaboration platform leveraging a modern technology stack, specifically React.js for the dynamic frontend and Node.js with Express for the scalable back end. The system is engineered for responsiveness and cross-device compatibility, ensuring a consistent, intuitive user experience across desktop and tablet interfaces. At the core of its functionality, the platform enables real-time collaborative code editing powered by sophisticated Operational Transformation (OT) algorithms, which allow multiple users to modify the same codebase simultaneously with automatic conflict resolution and synchronization.

To facilitate diverse development needs, CodeCollab incorporates secure, multi-language code execution through Docker-based containerization. This architecture provides isolated runtime environments for each project, ensuring that code execution is reproducible and securely sandboxed from the host system. Complementing this is a fully functional, browser-integrated terminal emulator that provides users with direct command-line access to their containerized environments for comprehensive project management.

Elevating the user experience beyond simple text editing, intelligent code assistance is provided through the integration of the Language Server Protocol (LSP). This delivers professional-grade features such as context-aware auto-completion, semantic "go-to-definition" navigation, and automated refactoring capabilities. Furthermore, an integrated team communication and chat system is embedded directly within the coding workspace, allowing users to collaborate instantly through direct messaging, code snippet sharing, and mentions, thereby eliminating the need for external communication tools.

The system offers support for over 10 distinct programming languages, catering to the diverse requirements of students learning fundamental syntax and professionals managing complex, polyglot projects in a complete online IDE setting. From an organizational perspective, the integrated Admin Panel enables administrators to maintain comprehensive oversight, allowing them to monitor active users, manage project hierarchies, configure system settings, and visualize platform analytics. To ensure sustainability and accessibility, CodeCollab introduces a freemium business model via a secure Payment and Subscription Management System. This system supports Free, Premium, and Institutional plans, seamlessly integrated with the Stripe API and PayPal SDK to handle transactions and access controls. Finally, a robust Security and Compliance framework underpins the entire system, employing JWT-based stateless authentication, Helmet.js for secure HTTP headers, CORS policies, and aggressive rate limiting to ensure full compliance with stringent data protection standards.

1.1.3 Technologies Used

The CodeCollab system is built on a carefully selected modern technology stack:

1. React.js 18.x with Redux Toolkit 2.x and Tailwind CSS 3.x component-based frontend with centralized state management and utility-first styling.
2. Monaco Editor (Latest) VS Code-based editor with syntax highlighting for 40+ languages, multi-cursor editing, and LSP integration.

3. xterm.js 5.x fully functional browser-based terminal emulator with collaborative sharing support.
4. Node.js 20 LTS with Express.js 4.x high-performance server runtime for RESTful API development and business logic.
5. Socket.io 4.x real-time bidirectional communication for collaborative editing, chat, and notification broadcasting.
6. MongoDB 7.x with GridFS flexible NoSQL document database for projects, files, user data, and collaboration logs.
7. Redis 7.x in-memory data store for session caching, pub/sub event broadcasting, and Socket.io horizontal scaling adapter.
8. Elasticsearch 8.x distributed full-text search engine for project-wide code search with regex support and incremental indexing.
9. Docker 24.x with Docker Compose 2.x containerization for secure, isolated multi-language code execution environments.
10. JWT with bcrypt stateless authentication tokens and password hashing for secure user authentication and authorization.
11. Stripe API / PayPal SDK secure payment gateway integration for freemium subscription management.
12. Jest + Cypress + ESLint + Prettier comprehensive testing and code quality assurance toolchain.

1.1.4 System Overview

CodeCollab operates as an interconnected, multi-tiered system supporting users at multiple levels. Regular users (developers and students) can create accounts, manage projects and file hierarchies, invite collaborators, assign granular permissions (Owner, Editor, Commenter, Viewer), and work collaboratively in real time through the Monaco-based code editor with LSP-powered intelligent assistance. The Docker-based execution engine processes code in isolated containers with configurable resource limits, preventing any single execution from affecting platform stability. The integrated xterm.js terminal supports multiple instances per project, command history, and collaborative sharing. The team communication system embeds group and private messaging directly in the workspace, supporting rich text formatting, inline code snippet sharing, mentions,

emoji reactions, and persistent searchable history. Collaborative debugging synchronizes breakpoints, variable inspection, and step-through controls across all project collaborators. The Git integration module supports commit, push, pull, and branch management directly within the browser. Administrators manage the entire platform through a centralized analytics dashboard with Chart.js and Recharts visualizations, covering user management, subscription oversight, execution metrics, and security monitoring.

1.1.5 Outcome of the Project

The CodeCollab project delivers a production-ready, collaborative development ecosystem that demonstrably improves the quality and efficiency of software development and programming education. By consolidating the functionality of multiple specialized tools into a single, coherent platform, CodeCollab eliminates the overhead of tool management and context-switching. The platform provides students and junior developers with access to professional-grade development tools without the installation complexity of desktop IDEs, reducing the barrier to entry for collaborative development in educational environments. The outcome is a secure, cloud-based digital environment where development workflows flow seamlessly, enabling developers to collaborate effectively and learners to access professional-grade tools without complex setup.

1.2 Relevance to Course Modules

The CodeCollab project is a comprehensive manifestation of the academic curriculum covered throughout the Bachelor of Science in Software Engineering program at COMSATS University Islamabad, Sahiwal Campus (2022-2026). Every architectural decision, implementation choice, and design pattern reflects concepts systematically studied across multiple course modules. The following subsections detail how each major course module directly informed the design and implementation of CodeCollab.

1.2.1 Software Engineering

Software Engineering formed the foundational framework for the entire development lifecycle of CodeCollab. Requirement elicitation techniques were applied to identify the needs of developers, students, and educators. The SRS and SDD document structures, use case modeling, functional and non-functional requirement categorization, and SDLC methodology selection all follow the systematic approach taught in Software Engineering. The Agile development methodology

adopted for the project with two-week sprints, product backlog management, and iterative delivery directly applies the Agile software process models covered in the course. The MVVM architectural pattern reflects design principles and patterns systematically studied in the curriculum.

1.2.2 Database Systems

Database Systems provided the knowledge required to design CodeCollab's multi-tier data storage architecture. Core database concepts including indexing, query optimization, data access patterns, and data integrity were adapted for MongoDB's document-oriented model. The schema design for CodeCollab's core entities (User, Project, File, CollaborationSession, ExecutionLog, Subscription, ChatMessage, AuditLog) reflects understanding of entity relationships and optimized data retrieval. The decision to use MongoDB GridFS for large file storage, Redis for high-frequency caching and pub/sub, and Elasticsearch for full-text search demonstrates application of database systems knowledge to select appropriate storage technology for each data access pattern.

1.2.3 Web Engineering and Internet Programming

Web Engineering and Internet Programming directly enabled development of CodeCollab's React.js frontend and Express.js back end. The course covered HTTP protocol fundamentals, RESTful API design principles, stateless communication, and browser-side JavaScript programming all foundational to CodeCollab's 50+ REST API endpoints. React.js component architecture, Redux Toolkit state management, client-side routing, and API integration patterns all reflect web development principles covered in this course.

1.2.4 Object Oriented Programming

OOP principles are evident throughout CodeCollab's architecture. The back end service layer is organized around domain-specific service classes AuthService, ProjectService, ExecutionService, CollaborationService, ChatService, NotificationService, PaymentService, AdminService, SecurityService each encapsulating domain logic and depending on Repository classes for data access. The frontend component architecture in React.js embodies modularity, reusability, and separation of concerns through functional components, custom hooks, and context providers.

1.2.5 Data Structures and Algorithms

The Operational Transformation algorithm at the core of the real-time collaboration engine is a complex, context-aware distributed algorithm that transforms concurrent operations to maintain document consistency a direct application of algorithm design principles. Elasticsearch's inverted index for full-text search, the execution queue's priority structure for managing concurrent code execution requests, and the file tree data structure for hierarchical project management all represent practical applications of data structures studied in this course.

1.2.6 Human Computer Interaction

HCI principles guided every interface design decision in CodeCollab. The VS Code-style layout with file explorer, code editor, output panel, and terminal mirrors widely validated IDE interaction patterns. Dark, light, and high-contrast theme options, keyboard shortcut customization, and responsive layout design reflect HCI principles of accessibility and personalization. Error messages with actionable suggestions, confirmation dialogs for destructive operations, and real-time visual feedback for collaboration state changes (cursor positions, selection highlights) demonstrate the application of usability principles.

1.2.7 Information Security

JWT-based stateless authentication, bcrypt password hashing, OAuth 2.0 social login, and role-based access control implement authentication and authorization principles from the Information Security course. Helmet.js configures HTTP security headers (Content-Security-Policy, X-Frame-Options, X-XSS-Protection). Rate limiting prevents brute-force and DDoS attacks. Docker container namespace isolation prevents code execution from affecting host systems. GDPR-aligned data handling ensures compliance with data protection regulations. Each security mechanism directly maps to threats and countermeasures studied in the course.

1.3 Project Background

The idea behind the CodeCollab project originates from the increasing need for a centralized and reliable digital development environment that supports real-time collaboration, especially in remote teams and educational settings. Traditional collaborative coding tools force users to switch between multiple platforms one for editing, another for communication, a third for version control,

and a separate environment for code execution. This fragmentation reduces productivity and makes collaborative development unnecessarily complex.

The project idea further developed by analyzing the challenges faced by programming students and development teams at COMSATS University Islamabad, Sahiwal Campus. Students attending programming classes often lack access to a shared environment where they and their instructors can see and edit the same code simultaneously. Remote development teams face similar challenges needing real-time synchronization, integrated communication, and collaborative debugging within a single workspace. The absence of such a platform in the local educational and professional context made the development of CodeCollab both practically valuable and academically relevant.

The specific features of CodeCollab were shaped by analyzing workflow gaps through online research on developer community platforms (Stack Overflow, Reddit, GitHub Discussions), studying the official documentation and limitation reports of existing platforms (CodePen, Replit, VS Code Live Share, CodeSandbox, GitHub Codespaces), and direct conversations with students and faculty at COMSATS University Islamabad. The findings consistently pointed to the same unmet needs: a browser-based collaborative editor supporting multiple programming languages, integrated team communication, collaborative debugging, and professional code intelligence all without requiring local installation.

1.4 Literature Review

The following comparative analysis reviews existing collaborative development platforms against CodeCollab's proposed feature set to establish the context and justification for the system's design decisions.

Table 1: Comparative Analysis of Existing Platforms and CodeCollab Solution

Platform	Core Features	Key Limitations	CodeCollab Solution
CodePen	Real-time HTML/CSS/JS preview, live collaboration, code forking	Only supports HTML/CSS/JS; no terminal, multi-language, or AI	Real-time collaboration, 20+ languages, terminal, team chat
Replit	Multi-language IDE, sharing, basic collaboration	Limited intelligence, no collaborative debugging, heavy	LSP-based intelligence, collaborative debugging, analytics

		resources	
VS Code Live Share	Real-time editing, shared debugging	Requires installation, no web support, no execution or chat	Fully web-based, built-in execution and communication
CodeSandbox	Web IDE, containers, templates	Performance issues, limited terminal, weak backend support	Multi-user optimized, full terminal, Docker execution
GitHub Codespaces	Browser-based VS Code, containers, GitHub integration	Expensive, limited free tier, slow startup, no chat	Fast, affordable, built-in chat, standalone platform

1.5 Analysis from Literature Review

1.5.1 Alignment with Real-Time Collaboration Trends

Research on real-time collaborative editing consistently identifies Operational Transformation (OT) and Conflict-free Replicated Data Types (CRDTs) as the primary approaches to achieving eventual consistency in distributed editing systems. CodeCollab implements OT over the WebSocket transport layer provided by Socket.io, following the architectural pattern established by Ellis and Gibbs (1989) and refined by Sun and Ellis (1998). The choice of OT is justified by its lower memory overhead for text-based documents and well-established convergence guarantees for sequential text operations the predominant operation type in code editing.

1.5.2 Validation of Technology Choices

Literature analysis validates each major technology decision in CodeCollab's stack. MongoDB's document model is well-suited to the variable structure of programming project data. Redis Pub/Sub has been validated as the appropriate mechanism for broadcasting real-time events across horizontally scaled Socket.io server instances, with documented delivery latency under 10 milliseconds for standard payloads. Docker containerization is widely recognized as the industry-standard approach to secure, isolated code execution in multi-tenant environments.

1.5.3 Gaps Addressed by CodeCollab

The literature review revealed consistent gaps in existing collaborative development platforms that CodeCollab specifically addresses: (1) the absence of integrated team communication requiring external tools; (2) the lack of collaborative debugging in browser-based IDEs; (3) language

support restrictions in web-based collaborative editors; (4) the installation barrier imposed by desktop-dependent collaboration tools; and (5) the absence of execution analytics and educational monitoring in collaborative platforms. CodeCollab's 18-module architecture directly addresses each identified gap.

1.5.4 Security and Code Execution Safety

Research on security risks in shared coding environments highlights the critical importance of code execution isolation. Academic and industry literature documents multiple instances of code execution vulnerabilities in web-based IDEs where insufficient container isolation allowed malicious code to access host system resources. CodeCollab's Docker-based execution engine addresses these concerns through namespace isolation, cgroup resource limits, filesystem sandboxing, and post-execution container destruction following best practices documented in Docker's official security guidelines.

1.5.5 Educational Technology Research

Research in educational technology consistently demonstrates that collaborative coding environments improve learning outcomes in programming education. Studies show that the ability to see peers' code in real time, receive immediate feedback from instructors, and engage in pair programming activities correlates with improved code quality, faster debugging, and better understanding of software design principles. CodeCollab's execution analytics and instructor monitoring features are directly informed by educational technology research on formative assessment in programming courses.

1.6 Methodology and Software Lifecycle

1.6.1 Agile Methodology

For the development of CodeCollab, the Agile methodology is adopted to ensure flexibility, continuous improvement, and iterative progress. The project is divided into short two-week sprints, allowing frequent testing, feedback collection, and rapid implementation of enhancements. GitHub Projects is utilized for task management through Kanban-style boards, enabling clear planning, assignment, and progress tracking throughout the development cycle. The Agile approach ensures early detection of errors, incremental delivery of modules, and ongoing feedback from the project supervisor.

1.6.2 MVVM Architecture

The system follows the Model-View-View Model (MVVM) architectural pattern to promote a clean separation between the user interface, data handling, and business logic. This structure enhances maintainability, scalability, and testing efficiency. The frontend is developed using React.js with a component-based architecture, where Redux Toolkit manages application state. The back end is designed using Node.js and Express.js, following RESTful API design principles with a clear separation between routes, controllers, services, and data models.

1.6.3 Sprint Structure

Development is organized into twelve two-week sprints across the 24-week project timeline. Sprint 1 covers authentication and database setup. Sprints 2-3 implement the code editor and real-time collaboration infrastructure. Sprint 4 builds the Docker-based execution engine. Sprint 5 implements LSP code intelligence. Sprints 6-7 develop project management, sharing, terminal, and live preview. Sprint 8 implements the team communication system. Sprint 9 builds collaborative debugging. Sprint 10 delivers Git integration and Elasticsearch search. Sprint 11 develops the admin panel and payment system. Sprint 12 implements notifications, security hardening, and final integration testing.

1.6.4 Team Work Division

Table 2: Team Members Work Division

Student Name	Registration Number	Role	Assigned Modules & Responsibilities
Ahmad Ali Raza	CIIT/FA22-BSE-062/SWL	Backend & Infrastructure Lead	Modules: 1 (Auth), 3 (Collaboration), 4 (File System), 5 (Sharing), 7 (Execution), 8 (Terminal), 13 (Analytics), 14 (Git), 15 (Admin Panel), 16 (Payment), 18 (Security) Additional: Backend API design (Node.js/Express.js), MongoDB/Redis architecture, Docker containerization, Socket.io infrastructure, authentication

			and security middleware
Faizan Mahmood	CIIT/FA20-BSE-044/SWL	Frontend & User Experience Lead	Modules: 2 (Editor), 6 (Live Preview), 9 (Code Intelligence), 10 (Chat), 11 (Debug), 12 (Search), 17 (Notifications) Additional: UI/UX design (React.js/Tailwind CSS), Monaco Editor integration, LSP bridge implementation, Redux Toolkit state management, responsive design, Cypress end-to-end testing

1.6.5 Testing and Quality Assurance

Testing is integrated throughout every sprint rather than being a terminal phase. Each sprint includes unit testing of new modules using Jest, integration testing of module interactions, and end-to-end testing of user workflows using Cypress. ESLint enforces code quality rules and Prettier ensures consistent formatting. Postman validates all REST API endpoints. The automated test suite is executed before merging any feature branch to prevent regressions. Security testing using OWASP ZAP is conducted before each major release.

Chapter 2

PROBLEM DEFINITION

2. Problem Definition

This chapter discusses the precise problem to be solved by the CodeCollab project. The core issue is the absence of a unified digital platform for collaborative software development that integrates real-time editing, code execution, team communication, version control, and project management into a single browser-accessible workspace. Currently, developers and students are forced to use multiple disconnected tools, resulting in context switching, productivity loss, and inefficient collaboration workflows.

Despite the advancements in web-based IDEs, developers and students continue to face significant challenges when collaborating on code projects. Most existing solutions are limited either in language support or in the depth of professional features they offer. While some platforms provide real-time editing, they often lack integrated debugging, team communication systems, or version control support. This fragmentation forces users to switch between multiple tools, resulting in reduced productivity, context switching, and inefficient teamwork.

Furthermore, current platforms rarely provide execution tracking, integrated chat functionality, or collaborative debugging all crucial for modern software learning and collaborative development. Educators struggle to monitor real-time student progress, while developers face difficulties in managing shared codebases efficiently and communicating effectively within their development environment.

2.1 Problem Statement

The healthcare and more broadly, the software development and educational community faces a fundamental challenge: the absence of a unified, browser-accessible collaborative development platform that integrates all essential development activities into a coherent workspace. Current solutions are either narrowly specialized, require local software installation, lack integrated team communication, or impose prohibitive costs for teams requiring advanced features.

Developers working in distributed or remote settings must assemble a fragmented toolchain: a browser-based editor for code editing, a separate communication platform (Slack, Discord, Microsoft Teams) for team interaction, a distinct version control interface (GitHub, GitLab) for code management, and additional tools for code execution and debugging. This fragmentation introduces significant productivity overhead, reduces the quality of real-time collaboration, and

makes it difficult to maintain a shared, coherent understanding of the codebase during collaborative sessions.

In educational settings, programming instructors face unique challenges that existing platforms do not adequately address. The inability to view and interact with students' code in real time makes remote programming instruction significantly less effective than in-person lab sessions. The absence of execution tracking and analytics prevents instructors from monitoring student progress, identifying common mistakes, or providing timely intervention. Therefore, there is a strong need for a unified collaborative platform that not only simplifies teamwork but also enhances learning, debugging, and code quality through integrated communication tools and comprehensive analytics.

2.2 Deliverables and Development Requirements

The CodeCollab project delivers a comprehensive set of components designed to address each identified gap in the current collaborative development landscape.

2.2.1 Deliverables

1. Collaborative Code Editor Web Application

A full-featured, browser-based code editor built on Monaco Editor with syntax highlighting for 40+ programming languages, multi-cursor editing, code folding, bracket matching, a minimap for navigation, split-view capabilities, and customizable themes including dark, light, and high-contrast modes. The editor integrates with the real-time collaboration infrastructure, enabling multiple users to edit the same file simultaneously with visible cursor positions and selection highlights for all active collaborators. Auto-save triggers every 5 seconds and on every file switch.

2. Multi-Language Code Execution Engine

A Docker-based code execution system supporting Python, JavaScript/Node.js, TypeScript, Java, PHP, C, C++, Go, Rust, and additional languages through pre-built Docker images. Each execution occurs in a fresh, isolated container with configurable resource limits (CPU ≤ 2 cores, RAM ≤ 1 GB, timeout 60 seconds). The execution engine supports package and dependency installation for each language ecosystem and streams output back to the client in real time via Socket.io.

3. Integrated Browser Terminal

A fully functional browser-based terminal emulator using xterm.js, providing shell command execution, command history with search functionality (Ctrl+R), auto completion, multiple

terminal instances per project (up to 5) with tab management, and collaborative terminal sharing for pair programming scenarios. The terminal connects to the Docker execution environment, enabling users to run arbitrary shell commands within the project's language context.

4. Team Communication System

A real-time team communication system embedded within the development workspace, supporting group project channels and private direct messages. Features include rich text formatting, inline code snippet sharing with syntax highlighting, file attachment sharing (up to 25 MB), mentions with notification triggering, emoji reactions, typing indicators, read receipts, message editing with visible edit history, and persistent searchable message archives retained for 90 days.

5. Administrative Panel

A centralized administrative dashboard for platform management providing tools for user account management (create, view, edit roles, suspend, delete), active session monitoring, platform analytics visualization using Chart.js and Recharts, Docker resource limit configuration, execution timeout management, subscription and revenue tracking, security alert monitoring, and audit log review. The admin panel is accessible exclusively to users with the Administrator role.

6. Payment and Subscription Management System

A secure subscription management system integrated with Stripe API and PayPal SDK, supporting Free, Premium, and Institutional subscription tiers. The system handles subscription creation, renewal, cancellation, invoice generation, receipt delivery, payment failure handling, and automatic role-based feature unlocking upon successful subscription upgrade. No sensitive payment data is stored on CodeCollab's servers all financial data is tokenized and managed by the payment gateway providers in compliance with PCI-DSS standards.

2.2.2 Development Requirements

Table 3: CodeCollab Technical Development Requirements

Category	Requirement	Specification / Version
Frontend Framework	React.js	18.x with concurrent rendering and automatic batching
State Management	Redux Toolkit	2.x with RTK Query for efficient API state management
CSS Framework	Tailwind CSS	3.x with JIT compiler for production-optimized

		styling
Code Editor	Monaco Editor	Latest stable VS Code editing engine with full LSP support
Terminal Emulator	xterm.js	5.x with WebGL renderer for hardware-accelerated terminal rendering
Backend Runtime	Node.js	20 LTS with native fetch API and improved startup performance
Backend Framework	Express.js	4.x with async/await error handling middleware chains
Real-Time Communication	Socket.io	4.x with binary data support, rooms, and namespaces
Primary Database	MongoDB	7.x with aggregation pipeline, GridFS, and change streams
Cache & Pub-Sub	Redis	7.x with Redis Stack for advanced data structures and Pub/Sub
Search Engine	Elasticsearch	8.x with inverted index and incremental update support
Containerization	Docker Engine	24.x with BuildKit for efficient image layer caching
Container Orchestration	Docker Compose	2.x for development and staging environment management
Authentication	JWT + bcrypt	RS256-signed tokens; bcrypt work factor 12 for password hashing
OAuth	Google + GitHub OAuth 2.0	Authorization Code Flow for social login integration
Payment	Stripe API + PayPal SDK	Latest stable with webhook signature verification
Security Middleware	Helmet.js + CORS + Rate Limiter	Security headers, CORS policy, per-IP request throttling
Testing	Jest + Cypress	Jest 29.x unit/integration; Cypress 13.x end-to-end

Code Quality	ESLint + Prettier	ESLint 8.x with security plugin; Prettier 3.x auto-formatting
UI Design	Figma	Wireframes, component library, mockups, and prototyping

2.3 Current System Analysis

A thorough analysis of the current landscape of collaborative development tools reveals systematic gaps that directly inform CodeCollab's feature set and architectural decisions. The following subsections examine each major limitation category in detail.

Table 4: Summary of Current System Limitations and CodeCollab Responses

Limitation	Specific Problem	Impact	CodeCollab Solution
Toolchain Fragmentation	5-8 separate tools required for complete development workflow	High context-switching cost; reduced collaboration quality	18-module unified platform edit, run, communicate, debug, version-control in one workspace
Language Restrictions	Most web IDEs limited to JS/HTML or inconsistent multi-language support	Forces use of local IDEs for back end/systems development	Docker-based execution supporting 10+ languages with consistent quality and isolation
No Collaborative Debugging	No browser IDE supports shared breakpoints and variable inspection	Remote pair debugging requires screen sharing or async workflow	Synchronized breakpoints, variable inspection, step-through for all collaborators via DAP
No Communication	No built-in chat; external tools	Context-switching to	Built-in group and DM chat with code snippets, mentions within

Integration	required	Slack/Discord; loss of code context in discussion	the workspace
No Educational Analytics	No execution tracking or instructor monitoring features	Instructors cannot assess student progress without direct observation	Per-user execution history, error analysis, instructor analytics dashboard
Installation Barriers	Desktop IDE extensions required for professional collaborative features	Inaccessible in restricted lab environments; cross-platform issues	Fully browser-based; no installation required; modern browser sufficient
Cost Barriers	Advanced collaboration features require expensive subscriptions	Inaccessible for students, educational institutions, small teams	Freemium model with meaningful free tier; affordable Premium and Institutional plans

Chapter 3

REQUIREMENT ANALYSIS

3. Requirement Analysis

The Requirement Analysis chapter provides a comprehensive and structured understanding of what the proposed CodeCollab system must accomplish in order to function effectively within the intended collaborative development and educational environments. It serves as a foundation for translating user expectations into clear technical requirements by identifying system behavior, user interactions, and the specific tasks the system must perform. This chapter outlines how CodeCollab is expected to operate, detailing all 18 use case descriptions, formal functional requirements (FR-1 through FR-18), and twelve non-functional requirement categories that collectively define the system's quality standards, performance targets, and compliance obligations.

3.1 Introduction to Requirement Analysis

Requirement Analysis is a critical phase in the software development lifecycle, serving as the foundation upon which the entire system is designed, implemented, and evaluated. For CodeCollab, this phase ensures that all functional expectations, performance needs, and operational constraints are clearly understood before development begins. The purpose of conducting requirement analysis is to bridge the gap between user needs and technical implementation, ensuring the final system effectively resolves existing collaborative development challenges such as fragmented toolchains, absent team communication, and missing collaborative debugging.

The requirement analysis process for CodeCollab combined structured analysis of existing platforms to identify feature gaps, review of academic research on OT algorithms and code execution security, informal interviews with students and developers at COMSATS University Islamabad, and iterative review sessions with the project supervisor SE Mr. Shahzad Ali. The resulting requirements follow the IEEE SRS standard format, organized into use case descriptions, functional requirements, and non-functional requirements.

3.2 Actors in the System

1. Regular User (Developer / Student)

Regular users are the primary consumers of CodeCollab collaborative development features. They interact with the system to create and manage projects, invite collaborators, edit code in real time,

execute programs in multiple languages, use the integrated terminal, communicate via team chat, debug collaboratively, manage Git repositories, and search project code. Regular users are subject to the resource and collaboration limits of their subscription tier (Free, Premium, or Institutional).

2. Educator / Instructor

Educators use CodeCollab to conduct live collaborative programming sessions, monitor student execution activity in real time, review execution history and error analytics across their class cohort, share code templates and starter projects, and provide in-editor feedback. Educators may be assigned a dedicated Educator role granting access to the instructor analytics dashboard.

3. Administrator

Administrators have comprehensive control over the CodeCollab platform. They manage user accounts, assign roles, monitor active sessions and system resource utilization, configure platform-wide settings, review security audit logs, manage subscription records, and generate analytics reports. All administrative actions are logged in the AuditLog collection for compliance purposes.

4. External Systems

CodeCollab interacts with several external systems: Authentication System (JWT/OAuth), Collaboration Server (Socket.io), Docker Engine for code execution, LSP Servers (Pyrite, TypeScript LS, clangd, gopls, rust-analyzer) for code intelligence, Git Server for version control, Payment Gateway (Stripe/PayPal) for transactions, and the System Database (MongoDB, Redis, Elasticsearch) for all data storage and retrieval.

3.3 Use Case Diagrams

The CodeCollab main use case diagram illustrates interactions between the primary actors (User, Educator, Admin) and the 18 functional modules. Users perform development activities (code editing, execution, collaboration, debugging), communication activities (chat, notifications), and project management activities (files, search, Git). Administrators interact with all management and monitoring functions. External systems support authentication, execution, code intelligence, version control, search, payment, and notification delivery.

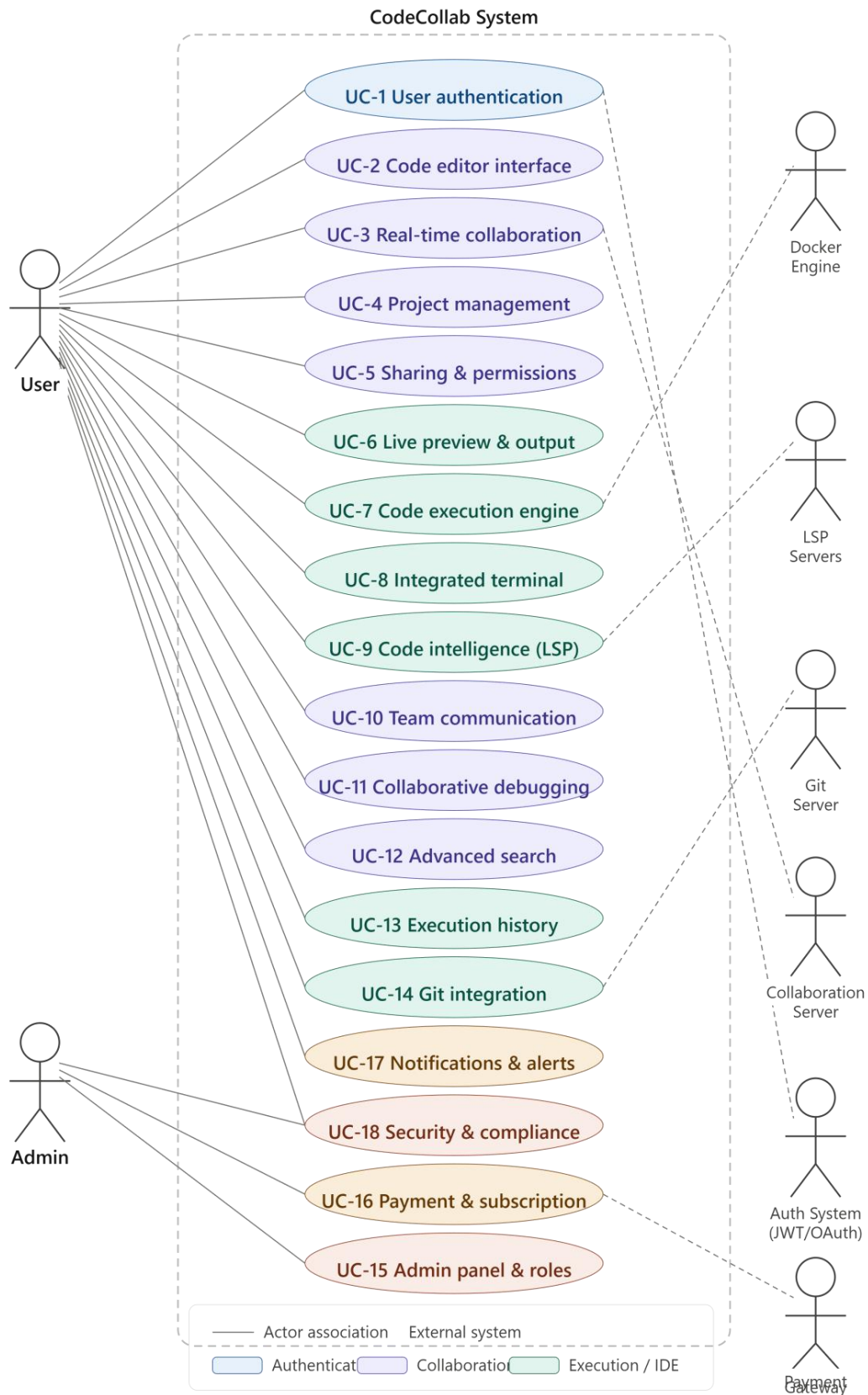


Figure 1: Use Case Diagrams

3.4 Use Case Descriptions

The following 18 tables provide structured descriptions of each use case in the CodeCollab system. Each table follows the standardized format covering Use Case ID, Name, Actors, Description, Trigger, Preconditions, Postconditions, Normal Flow, Alternative Flow, Exceptions, Business Rules, and Assumptions as defined in the SRS document.

UC-1: User Authentication & Profile Management	
Use Case ID	UC-1
Name	User Authentication & Profile Management
Actors	User, Authentication System (JWT/OAuth)
Description	Allows users to register, log in, and manage their profile using email/password or OAuth (Google, GitHub). Supports JWT-based session management, password reset, and profile editing including avatar and bio.
Preconditions	User has a valid email address and internet connectivity.
Postconditions	User is authenticated and granted access to the platform dashboard.
Normal Flow	1. User navigates to login/register page. 2. User enters credentials or selects OAuth provider. 3. System validates credentials via JWT/OAuth. 4. Successful authentication redirects user to dashboard. 5. Session token stored securely in browser.
Alternative Flow	• A1: OAuth login — user redirected to provider, token returned on success. • A2: Forgotten password — system sends reset link via email.
Exceptions	• E1: Invalid credentials — error message displayed, account locked after 5 attempts. • E2: OAuth provider unavailable — fallback to email login suggested.

Business Rules	• BR-1.1: Passwords must be minimum 8 characters with uppercase and number. • BR-1.2: JWT tokens expire after 24 hours; refresh tokens valid 7 days. • BR-1.3: Email must be verified before full access is granted.
-----------------------	--

UC-2: Code Editor Interface	
Use Case ID	UC-2
Name	Code Editor Interface
Actors	User
Description	Provides a full-featured browser-based code editor powered by Monaco Editor. Supports syntax highlighting, multi-tab editing, file tree navigation, theme switching (light/dark), and keyboard shortcuts matching VS Code conventions.
Preconditions	User is authenticated and has an active project open.
Postconditions	Code changes are saved to the project file system.
Normal Flow	1. User opens a project file from the file tree. 2. Editor loads file with appropriate syntax highlighting. 3. User edits code; changes auto-saved every 30 seconds. 4. User can open multiple files in tabs. 5. User applies theme and font size preferences.
Alternative Flow	• A1: File not found — editor shows placeholder with option to create file. • A2: Unsaved changes on close — confirmation dialog shown.
Exceptions	• E1: Editor fails to load — fallback plain-text editor shown. • E2: File too large (>10MB) — warning shown, read-only mode applied.

Business Rules	• BR-2.1: Auto-save triggered every 30 seconds or on focus loss. • BR-2.2: Maximum 20 tabs open simultaneously per session. • BR-2.3: File size limit 10MB per file.
-----------------------	--

UC-3: Real-Time Collaboration System	
Use Case ID	UC-3
Name	Real-Time Collaboration System
Actors	User, Collaboration Server (Socket.io)
Description	Enables multiple users to simultaneously edit the same file using Operational Transformation. All participants see each other's cursors, selections, and changes in real time with sub-100ms latency.
Preconditions	User is authenticated and has been invited to a shared project.
Postconditions	All collaborators' changes are merged and persisted without conflicts.
Normal Flow	1. User opens a shared project link. 2. System establishes WebSocket connection via Socket.io. 3. Collaborators' cursors appear with distinct colors and usernames. 4. User edits code; changes broadcast to all active collaborators. 5. Operational Transformation resolves concurrent edit conflicts.
Alternative Flow	• A1: Collaborator disconnects — ghost cursor removed after 5 seconds. • A2: Conflict detected — OT algorithm merges changes automatically.
Exceptions	• E1: WebSocket connection lost — system attempts reconnect 3 times then shows offline banner. • E2: Session timeout after 30 minutes of inactivity.
Business Rules	• BR-3.1: Maximum 10 concurrent collaborators per file. • BR-3.2: Changes must sync within 100ms under normal network conditions.

UC-4: Project Management & File System	
Use Case ID	UC-4
Name	Project Management & File System
Actors	User
Description	Allows users to create, organize, rename, and delete projects and files. Supports folder hierarchies, drag-and-drop file organization, project templates, and bulk file operations.
Preconditions	User is authenticated and on the dashboard.
Postconditions	Project or file changes are persisted in MongoDB and reflected in the file tree.
Normal Flow	1. User creates a new project and selects a template. 2. System generates project structure and file tree. 3. User creates, renames, moves, or deletes files and folders. 4. Changes reflected instantly in the file tree panel. 5. Project metadata (name, language, description) saved to database.
Alternative Flow	• A1: Template selected — pre-populated files and folders created. • A2: Import from ZIP — files extracted and project created.
Exceptions	• E1: Duplicate project name — user prompted to rename. • E2: Storage quota exceeded — upload blocked, upgrade prompt shown.
Business Rules	• BR-4.1: Project names must be unique per user account. • BR-4.2: Maximum 500 files per project on free tier. • BR-4.3: Deleted projects moved to trash; permanently removed after 30 days.

UC-5: Sharing & Permission Management	
Use Case ID	UC-5
Name	Sharing & Permission Management
Actors	User (Project Owner or Admin), Other Collaborators
Description	Allows users to share projects using secure links or email invitations with granular permission levels. Supports four roles: Owner, Editor, Commenter, Viewer. Permissions can be set at project and file levels. All access changes are logged in a permission audit trail.
Preconditions	User is the project Owner or has Admin privileges.
Postconditions	Invited members receive access at specified permission level; audit log entry created.
Normal Flow	1. Owner opens Share dialog and enters email and selects permission level. 2. System sends invitation email with secure acceptance link. 3. Invitee authenticates and accepts invitation. 4. MongoDB updated with collaborator permission record. 5. Collaborator appears in project collaborators list.
Alternative Flow	• A1: Shareable link generates time-limited (72h) public link with specified permission. • A2: Revoke access — owner removes collaborator; all active sessions terminated for that user on project.
Exceptions	• E1: Invalid email address — validation error shown. • E2: Invitation link expired — invitee shown expiry message; owner notified to resend.
Business Rules	• BR-5.1: Permission hierarchy: Owner > Editor > Commenter > Viewer. • BR-5.2: Projects can have only 1 Owner; ownership transferable.

UC-6: Live Preview & Output	
Use Case ID	UC-6
Name	Live Preview & Output
Actors	User
Description	Provides a real-time preview panel for web projects (HTML/CSS/JS) that updates on save. Supports split-screen editor and preview, mobile viewport simulation, and console output display.
Preconditions	User has an active web project open in the editor.
Postconditions	Preview panel reflects latest saved code state.
Normal Flow	1. User writes HTML/CSS/JS code in the editor. 2. On save, preview panel auto-refreshes. 3. User can resize editor/preview split. 4. Console errors displayed in the output panel. 5. User can toggle mobile/tablet/desktop viewport sizes.
Alternative Flow	• A1: Manual refresh — user clicks Refresh button. • A2: External URL preview — user enters URL to preview external page.
Exceptions	• E1: JS runtime error — error highlighted in console with line reference. • E2: Infinite loop detected — execution halted after 5 seconds with warning.
Business Rules	• BR-6.1: Preview auto-refresh on every save event. • BR-6.2: Console output limited to 1000 lines to prevent memory overflow. • BR-6.3: Mobile viewport simulation uses standard breakpoints (320px, 768px, 1024px).

UC-8: Admin Panel & Role Management

Use Case ID	UC-15
Name	Admin Panel & Role Management
Actors	Administrator
Description	Provides a centralized administrative control center. Administrators view and manage all user accounts (create, view, edit roles, suspend, delete), monitor active collaboration sessions and system resource utilization, configure platform-wide settings (Docker limits, execution timeouts, subscription tier definitions), review security audit logs, manage subscription and payment records, and generate platform analytics reports with Chart.js/Recharts visualizations.
Trigger	Administrator logs in and accesses the Admin Dashboard.
Preconditions	User has Administrator role; JWT token validated with admin scope.
Postconditions	Administrative action applied across system; AuditLog entry created.
Normal Flow	1. Admin authenticates; admin panel dashboard loads. 2. Dashboard shows active users, active sessions, execution queue depth, system resource utilization. 3. Admin navigates to User Management; views paginated user list with search and filter. 4. Admin selects action (view, edit role, suspend, delete). 5. System applies change; notifies affected user if applicable. 6. AuditLog entry created with admin ID, action, target user, timestamp.
Alternative Flow	A1: Bulk operations admin selects multiple users; applies role change or suspension in batch. A2: Analytics export admin exports usage reports in CSV/PDF format.

Exceptions	E1: Database write failure operation rolled back; error logged; admin notified. E2: Admin attempting to delete own account blocked with explanatory message.
Business Rules	BR-15: Only Admins can modify roles or manage system settings. BR-15.2: All admin actions logged with full context in AuditLog. BR-15.3: Admin cannot modify or delete AuditLog entries.
Assumptions	A1: Admin account exists and JWT token validated with admin scope.

UC-9: Payment & Subscription Management System

Use Case ID	UC-16
Name	Payment & Subscription Management System
Actors	User, Admin, Payment Gateway (Stripe / PayPal)
Description	Handles CodeCollab's freemium model, enabling users to upgrade to Premium or Institutional plans through Stripe and PayPal payment gateways. Manages subscription creation, renewal, cancellation, payment failure handling, invoice generation, receipt delivery, and automatic role-based feature unlocking. No sensitive payment data stored on CodeCollab servers; all tokenized via payment gateways.
Trigger	User selects a paid plan or initiates a subscription upgrade.
Preconditions	User authenticated; payment gateway reachable; selected plan available.
Postconditions	User subscription tier updated in MongoDB; user role adjusted; invoice generated and emailed; webhook confirmed.
Normal Flow	1. User selects subscription tier. 2. Redirected to payment gateway-hosted checkout (PCI-compliant). 3. User completes payment on gateway page. 4. Gateway sends webhook; CodeCollab validates

	signature. 5. MongoDB subscription record created/updated; user role permissions updated. 6. Invoice generated; confirmation email sent.
Alternative Flow	A1: Payment failure user notified; 3 retry attempts before 24-hour lockout. A2: Subscription cancellation access continues until period end; no proration refund.
Exceptions	E1: Gateway timeout pending status; webhook retry handles eventual consistency. E2: Duplicate subscription idempotency key prevents duplicate charges.
Business Rules	BR-16: No sensitive payment data stored on CodeCollab servers. BR-16.2: Role upgrade only on confirmed payment transactions atomic. BR-16.3: Invoice generated for every successful transaction; accessible 5 years.
Assumptions	A1: Stripe/PayPal APIs operational and webhook endpoints reachable.

UC-10: Real-Time Notification & Alert System

Use Case ID	UC-17
Name	Real-Time Notification & Alert System
Actors	User, Admin
Description	Delivers real-time in-app notifications for all system and collaboration events: project invitations, mentions in chat, code execution completions, Git operation results, payment confirmations, subscription renewals, and administrative alerts. Delivered via Socket.io for connected users and persisted in MongoDB for offline users. Users manage notifications through a dedicated panel with mark-as-read, delete, filter, and per-category preference controls.

Trigger	Any system event with notification significance (invitation, mention, execution complete, payment, admin action).
Preconditions	Notification service active; Redis Pub/Sub connected; Socket.io server running.
Postconditions	Notification delivered within 300ms for connected users; stored for offline delivery; email sent for high-priority events.
Normal Flow	1. System event occurs. 2. Notification payload published to Redis channel for target user/group. 3. Socket.io server subscribes to channel; forwards to connected client via WebSocket. 4. In-app notification badge incremented; notification panel updated. 5. For high-priority events (security alerts, payment failures): email also sent.
Alternative Flow	A1: User offline notification stored in MongoDB; delivered within 5s of reconnection. A2: User muted category notification stored but no visual alert shown.
Exceptions	E1: Notification delivery timeout stored; retry on next connection. E2: Email delivery failure logged; in-app notification still delivered.
Business Rules	BR-17: Notifications delivered within 300ms for connected users. BR-17.2: Notifications retained 30 days; auto-deleted after retention period. BR-17.3: Users configure per-category notification preferences.
Assumptions	A1: Internal notification module (Redis/Socket.io) configured and running properly.

UC-11: Security & Compliance System

Use Case ID	UC-18
-------------	-------

Name	Security & Compliance System
Actors	User, Admin
Description	Implements platform-wide security including HTTPS enforcement, Helmet.js HTTP security headers (CSP, HSTS, X-Frame-Options), CORS policy enforcement, rate limiting (DDoS and brute-force prevention), JWT token validation on all protected endpoints, RBAC enforcement, API access token management, comprehensive audit logging of all security-relevant events, and GDPR-aligned data handling including right-to-erasure support.
Trigger	Any HTTP/WebSocket request to CodeCollab APIs or any security-relevant system event.
Preconditions	Security middleware stack configured; TLS certificates valid; rate limit counters initialized in Redis.
Postconditions	Request processed securely if authorized; unauthorized requests blocked; all events logged in AuditLog.
Normal Flow	1. Incoming request arrives at Express.js server. 2. Helmet.js adds security headers; CORS validates request origin. 3. Rate limiter checks Redis counter for IP/user; blocks if limit exceeded (429). 4. JWT validation extracts and validates token; populates request context with user identity and role. 5. RBAC checks user role against endpoint permission requirements. 6. Request processed; response returned. 7. Security event logged to AuditLog.
Alternative Flow	A1: Token expired 401 Unauthorized; client initiates token refresh flow. A2: Suspicious activity pattern alert generated; admin notified.
Exceptions	E1: Rate limit exceeded 429 Too Many Requests with Retry-After header. E2: SSL certificate expiry automated renewal alert 30 days before expiry.

Business Rules	BR-18: All sensitive endpoints require valid JWT and HTTPS. BR-18.2: Rate limits: 100 requests/min unauthenticated; 500/min authenticated. BR-18.3: AuditLog entries immutable admin cannot modify or delete. BR-18.4: GDPR compliance: data export and deletion requests processed within 30 days.
Assumptions	A1: Security middleware (Helmet.js, CORS, rate-limiter) configured and operational internally.

3.5 Functional Requirements

The following functional requirements formally specify the capabilities each CodeCollab module must deliver. Each requirement is defined with identifier, title, requirement statement, rationale, dependencies, and priority classification.

1. User Authentication & Profile Management

- **Requirement:** The system shall allow users to register, log in, and manage their profiles using JWT-based authentication with both email/password and OAuth (Google, GitHub) options. It shall support MFA and session management across devices.
- **Rationale:** To ensure secure user access and personalized configuration across the platform.
- **Dependencies:** None (foundational requirement)
- **Priority:** High

2. Code Editor Interface

- **Requirement:** The system shall provide a Monaco-based editor supporting syntax highlighting for 40+ languages, multi-cursor editing, split views, and customizable themes (light, dark, high contrast).
- **Rationale:** To offer a professional, user-friendly coding interface similar to modern IDEs that reduces the learning curve for developers.
- **Dependencies:** FR-1
- **Priority:** High

3. Real-Time Collaboration System

- **Requirement:** The system shall enable multiple users to collaboratively edit and view files in real time using Socket.io and Operational Transformation algorithms for conflict-free updates. Edit synchronization latency shall not exceed 200ms.
- **Rationale:** To facilitate teamwork and simultaneous project development with visible presence indicators.
- **Dependencies:** FR-1, FR-2
- **Priority:** High

4. Project Management & File System

- **Requirement:** The system shall provide file and folder management including creation, renaming, moving, duplication, and deletion, with autosave and drag-and-drop capabilities. Total project size capped at 500 MB; individual files at 10 MB.
- **Rationale:** To ensure organized management of project resources and provide an efficient workspace structure.
- **Dependencies:** FR-1
- **Priority:** High

5. Sharing & Permission Management

- **Requirement:** The system shall enable project owners to share projects via secure links or email invitations with four roles: Owner, Editor, Commenter, Viewer. Permissions shall be applied at both project and file levels.
- **Rationale:** To ensure controlled collaboration and security of shared content with auditable access trails.
- **Dependencies:** FR-1, FR-4
- **Priority:** High

6. Live Preview & Output

- **Requirement:** The system shall render real-time output for HTML/CSS/JavaScript projects and display console outputs for all supported languages. It shall support auto-refresh (minimum 500ms) and manual refresh modes.
- **Rationale:** To provide immediate feedback and faster iteration cycles for developers and students.

- **Dependencies:** FR-2, FR-7
- **Priority:** Medium

7. Team Communication & Chat System

- **Requirement:** The system shall support group and private chat channels with features such as mentions, emoji reactions, message editing, and persistent searchable history (90-day retention).
- **Rationale:** To streamline team collaboration without requiring external communication tools.
- **Dependencies:** FR-3
- **Priority:** Medium

8. Admin Panel & Role Management

- **Requirement:** The system shall provide an Admin Panel enabling administrators to manage user accounts, assign roles (Admin, Educator, Premium, User), monitor sessions, configure settings, review audit logs, and generate analytics reports.
- **Rationale:** To enable platform governance, security enforcement, and operational oversight of the multi-user production platform.
- **Dependencies:** FR-1, FR-5
- **Priority:** High

9. Payment & Subscription Management System

- **Requirement:** The system shall integrate with Stripe API and PayPal SDK to manage Free, Premium, and Institutional tiers. It shall handle payment processing, webhook-based subscription updates, invoice generation, and role-based feature unlocking. No sensitive payment data stored on CodeCollab servers.
- **Rationale:** To enable sustainable platform operation through the freemium model while maintaining PCI-DSS compliance.
- **Dependencies:** FR-1, FR-15
- **Priority:** High

10. Real-Time Notification & Alert System

- **Requirement:** The system shall deliver in-app and email notifications for project invitations, mentions, execution completions, payment events, and security alerts within 300ms via Socket.io and Redis Pub/Sub.
- **Rationale:** To keep users informed of platform events without requiring constant manual checking.
- **Dependencies:** FR-3, FR-10
- **Priority:** Medium

11. Security & Compliance System

- **Requirement:** The system shall implement HTTPS enforcement, Helmet.js security headers, CORS policy enforcement, rate limiting (100 req/min unauthenticated, 500 req/min authenticated), JWT validation on all protected endpoints, RBAC enforcement, and GDPR-compliant data handling including right-to-erasure.
- **Rationale:** Platform-wide security is non-negotiable for a collaborative platform handling user code, personal data, and payment information.
- **Dependencies:** FR-1, FR-15
- **Priority:** High

3.6 Non-Functional Requirements

The non-functional requirements define the quality attributes that CodeCollab must exhibit across all its modules. These requirements establish measurable performance targets, reliability guarantees, security standards, and operational constraints.

1. Performance

Code editing update propagation to all collaborators shall not exceed 200ms under normal network conditions. API response time for standard CRUD operations shall be $\leq 300\text{ms}$ at p95. Elasticsearch full-text search for projects up to 1,000 files shall respond within $\leq 500\text{ms}$ at p95. Real-time notification delivery shall occur within $\leq 300\text{ms}$ for connected users. The platform shall

support minimum 500 concurrent active projects without performance degradation, maintaining up to 15 active collaborators per project within the latency target.

2. Reliability

System uptime shall be $\geq 99\%$ excluding scheduled maintenance windows (announced 48 hours in advance). Automatic reconnection with full state recovery after network interruption shall complete within ≤ 5 seconds with no data loss for changes made during interruption. Daily automated backups of MongoDB and Redis data shall maintain a Recovery Point Objective (RPO) of ≤ 24 hours. Recovery Time Objective (RTO) after major failure shall be ≤ 4 hours with backup restoration.

3. Security

HTTPS enforced across all endpoints; HTTP requests redirected with HSTS header.

JWT tokens signed with RS256; access tokens expire in 24 hours; refresh tokens in 30 days.

Passwords hashed with bcrypt at work factor 12; raw passwords never stored or logged.

- Docker execution containers provide full namespace isolation; no shared filesystem or network with host.
- RBAC enforced at every API endpoint; unauthorized requests return 403 with no information leakage.
- Rate limiting: 100 req/min unauthenticated (IP-based); 500 req/min authenticated (user+IP).
- GDPR compliance: user data export within 30 days of request; account deletion within 30 days.
- Security audit logs retained for 1 year; immutable no admin modification permitted.

4. Usability

The VS Code-style interface reduces the learning curve for users familiar with mainstream IDEs. No installation is required; full functionality accessible from Chrome ≥ 110 , Firefox ≥ 110 , and Edge ≥ 110 . Dark, light, and high-contrast themes support accessibility across visual environments. Keyboard shortcuts for all common operations, customizable through user profile settings. Responsive layout optimized for desktop ($\geq 1280\text{px}$) and tablet ($\geq 768\text{px}$) screen widths.

5. Scalability

Horizontally scalable Socket.io cluster via Redis adapter new server instances addable without restart. MongoDB sharding supports data tier horizontal scaling. Docker execution containers dynamically allocated from queue with no fixed pool limit. Elasticsearch cluster scalable with

additional nodes for growing data volumes. Stateless REST API layer (JWT-based) enables horizontal scaling behind a load balancer without session affinity requirements.

6. Maintainability

MVVM architecture enforces separation of concerns; UI, state, and data access layers independently testable. Automated test suite (Jest + Cypress) with $\geq 70\%$ code coverage executed on every pull request. ESLint and Prettier enforce consistent code style across the entire codebase. Modular back end service architecture enables individual module replacement without cascading changes. Comprehensive JSDoc inline documentation and OpenAPI specification for all REST endpoints.

7. Portability

Browser-based platform runs on Windows 10+, macOS 10.15+, and Ubuntu 18.04+ without installation. Docker Compose configuration enables local development environment setup in under 10 minutes. Platform-agnostic REST API design enables future mobile application clients without backend changes.

8. Availability

Redundant MongoDB replica set (minimum 3 nodes) prevents single-point-of-failure data unavailability. Redis Sentinel or Redis Cluster provides high-availability caching and pub/sub. Health check endpoints for all critical services with automated alerting on service degradation. Admin panel and payment system maintain $\geq 99.5\%$ availability.

9. Compliance and Data Protection

All user data encrypted at rest (AES-256) and in transit (TLS 1.3). Payment data tokenized by Stripe/PayPal no card numbers or CVVs stored on CodeCollab servers. Audit logs retained for 1 year to support compliance investigations. User data export and deletion requests honored within 30 days per GDPR Article 17. Privacy policy and terms of service clearly accessible and versioned.

10. Notification Responsiveness

In-app notifications delivered within 300ms via Socket.io/Redis Pub/Sub for connected users. Email notifications queued and delivered within 5 minutes via SMTP. Offline notifications queued in MongoDB and delivered within 5 seconds of user reconnection.

11. Transaction Reliability

Payment transactions verified by webhook callback before subscription status is updated no premature role upgrades. Subscription state transitions are atomic using MongoDB sessions with full rollback on failure. Idempotency keys prevent duplicate charges from network retries.

12. Administrative Auditability

Every administrative action (user role change, suspension, deletion, system configuration change) logged with timestamp, admin ID, target entity, action type, and old/new values. Audit logs accessible only to Administrator role in read-only mode. Monthly audit log exports available for compliance review and investigation.

Chapter 4

DESIGN AND ARCHITECTURE

4. Design and Architecture

The Design and Architecture chapter provides a comprehensive technical blueprint of the CodeCollab system, detailing how its components are structured, how they interact, and how data flows through the platform to deliver real-time collaborative development capabilities. This chapter covers the system architecture, data representation, process workflows, and design models that collectively define CodeCollab technical foundation, reflecting the decisions recorded in the SDD (Scope Document) and informed by the requirements established in the SRS.

4.1 System Architecture

4.1.1 Architecture Style

CodeCollab is built on a three-tier, client-server architecture following the MVVM (Model-View-View Model) design pattern. The architecture separates concerns across three distinct tiers: the Presentation Tier (React.js frontend), the Business Logic Tier (Node.js/Express.js back end with Socket.io), and the Data Tier (MongoDB, Redis, Elasticsearch). This separation enables independent scaling of each tier, facilitates parallel development, and supports modular testing of individual layers.

Within the Business Logic Tier, a service-oriented architecture organizes functionality into domain-specific services: AuthService, ProjectService, CollaborationService, ExecutionService, ChatService, NotificationService, PaymentService, AdminService, and SecurityService. Each service encapsulates its domain logic, exposes interfaces to API controllers, and depends on Repository classes for data access. React.js components form the View, Redux Toolkit store serves as the ViewModel, and the back end service/repository layer implements the Model.

Real-time communication introduces a fourth architectural dimension alongside the standard request-response pattern: the WebSocket event layer. Socket.io manages persistent WebSocket connections between clients and the back end, routing collaboration events, chat messages, and notifications through dedicated namespaces. The Redis adapter extends this to multi-server deployments, ensuring events are routed correctly across horizontally scaled back end instances.

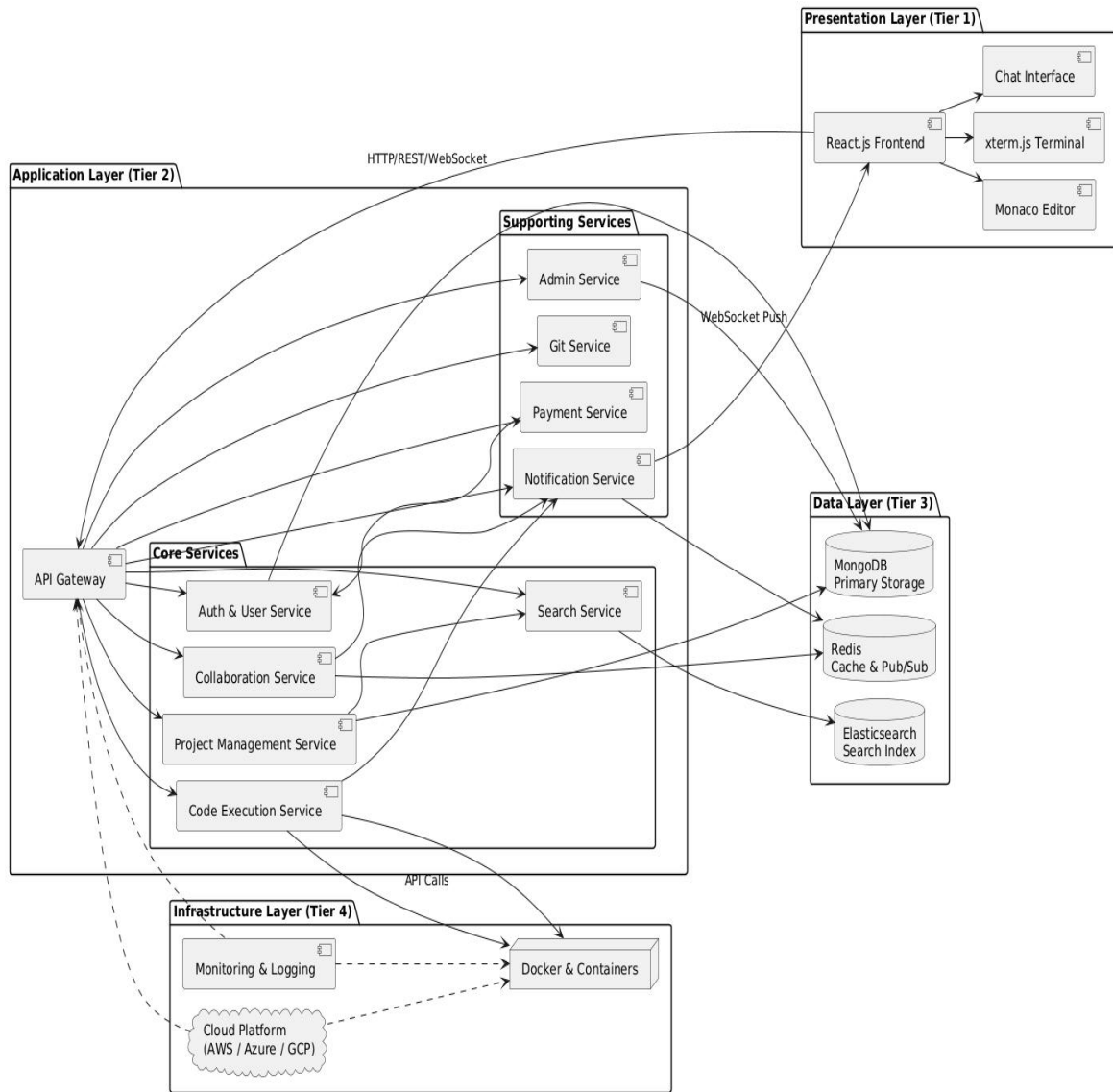


Figure 2: Architecture Style

4.1.2 System Architecture Components

Table 5: CodeCollab System Architecture Component Summary

Component	Technology	Tier	Responsibility
React.js UI	React.js 18.x + Redux Toolkit + Tailwind CSS	Presentation	User interface, state management, API consumption, WebSocket event handling

Monaco Editor	Monaco Editor (Latest)	Presentation	Code editing surface, syntax highlighting, LSP integration, OT delta generation
xterm.js Terminal	xterm.js 5.x	Presentation	Browser-based terminal rendering, terminal session management
Express.js API	Node.js 20 + Express.js 4.x	Business Logic	RESTful API routing, middleware orchestration, request validation
Socket.io Server	Socket.io 4.x	Business Logic	WebSocket connection management, real-time event routing and broadcasting
OT Engine	Custom Node.js module	Business Logic	Operational Transformation processing for conflict-free concurrent editing
Auth Service	JWT + bcrypt + OAuth 2.0	Business Logic	Token issuance/validation, password hashing, OAuth flow management
Execution Queue	Bull Queue + Redis	Business Logic	Code execution request queuing, priority management, Docker worker dispatch
Docker Workers	Docker Engine 24.x	Business Logic	Language-specific execution containers with resource-limited sandboxed execution
LSP Bridge	Node.js LSP WebSocket Proxy	Business Logic	Forwarding LSP JSON-RPC messages between Monaco Editor and language servers
MongoDB	MongoDB 7.x + GridFS	Data	Primary document store: users, projects, files, logs, messages,

			subscriptions
Redis	Redis 7.x	Data	Session cache, Socket.io adapter, Bull queue back end, Pub/Sub for notifications
Elasticsearch	Elasticsearch 8.x	Data	Full-text project search index with incremental indexing via MongoDB Change Streams
Language Servers	Pyrite, TypeScript LS, clangd, gopls, rust-analyzer	Service	Per-language code intelligence processing

4.2 Data Representation

4.2.1 Database Design and Schema

CodeCollab uses MongoDB 7.x as its primary data store. The flexible document model accommodates the variable-structure data generated by a multi-language collaborative development platform. The following table summarizes the core MongoDB collection schemas as defined in the SDD:

Table 6: Core MongoDB Collection Schema Reference

Collection	Key Fields	Relations hips	Purpose
users	_id, email, passwordHash, displayName, role, subscriptionTier, subscriptionExpiry, oauthProviders[], preferences {theme,shortcuts,notifications}, createdAt, lastActiveAt	Root entity	User profiles, credentials, preferences, subscription status
projects	_id, ownerId, name, description, language, template, collaborators [{userId,role}], rootFileId, createdAt, updatedAt,	users (ownerId, collaborat	Project metadata, collaborator lists, and workspace

	storageUsed	ors.userId)	settings
files	_id, projectId, parentId, name, type (file/folder), content (≤10MB text) or gridsId (>10MB), language, size, createdBy, updatedAt	projects (projectId); self-referential (parentId)	Hierarchical project file system with content storage
collaboration_sessions	_id, projectId, activeUsers[{userId,cursorLine,cursorCol,color}], otVersion, otHistory[], lastActivityAt	projects, users	Real-time OT state and collaborator presence tracking
execution_logs	_id, userId, projectId, language, codeHash, exitCode, stdoutTrunc, stderrTrunc, durationMs, memoryMB, errorType, timestamp	users, projects	Code execution audit trail and analytics data source
chat_messages	_id, projectId, channelType (group/dm), senderId, receiverId, content, contentType, attachments[], reactions[], editHistory[], timestamp, readBy[]	projects, users	Team communication message store with full history
subscriptions	_id, userId, tier, status, gatewayCustomerId, gatewaySubscriptionId, currentPeriodStart, currentPeriodEnd, invoices[]	users	Subscription lifecycle and payment reference tracking
notifications	_id, userId, type, title, body, payload, isRead, createdAt, expiresAt	users	Per-user notification queue for in-app and offline delivery
audit_logs	_id, adminId, actionType, targetEntityType, targetEntityId, previousValue, newValue,	users (adminId)	Immutable administrative

	ipAddress, timestamp		action audit trail for compliance
--	----------------------	--	-----------------------------------

4.2.2 Entity-Relationship Representation

The entity relationship model defines how core domain entities relate within MongoDB's document model. Users own Projects. Projects contain Files organized in a hierarchical tree using parent-child references. CollaborationSessions link multiple Users to a Project, tracking real-time OT state and cursor positions. ExecutionLogs associate with both the User who ran the code and the Project it ran from. ChatMessages belong to Projects and are authored by Users. Subscriptions are one-to-one with Users. AuditLogs record Administrator actions referencing the Administrator User as actor and the affected entity as target.

4.3 Process Flow

4.3.1 User Authentication and Session Management

The authentication process begins when a user submits credentials to POST /api/auth/login. The AuthService retrieves the user document from MongoDB by email, validates the password against the stored bcrypt hash using bcrypt.compare(), and checks account status (active, suspended, MFA-required). Upon successful validation, the service generates a JWT access token (RS256, 24h expiry) and a refresh token (RS256, 30d expiry), stores the refresh token hash in Redis (TTL 30d), and returns both tokens to the client. For OAuth flows, the authorization code is exchanged with the provider's token endpoint, user profile fetched, email matched to existing accounts or new account created, and JWT tokens issued. Refresh Token Rotation invalidates used refresh tokens to detect and respond to token theft.

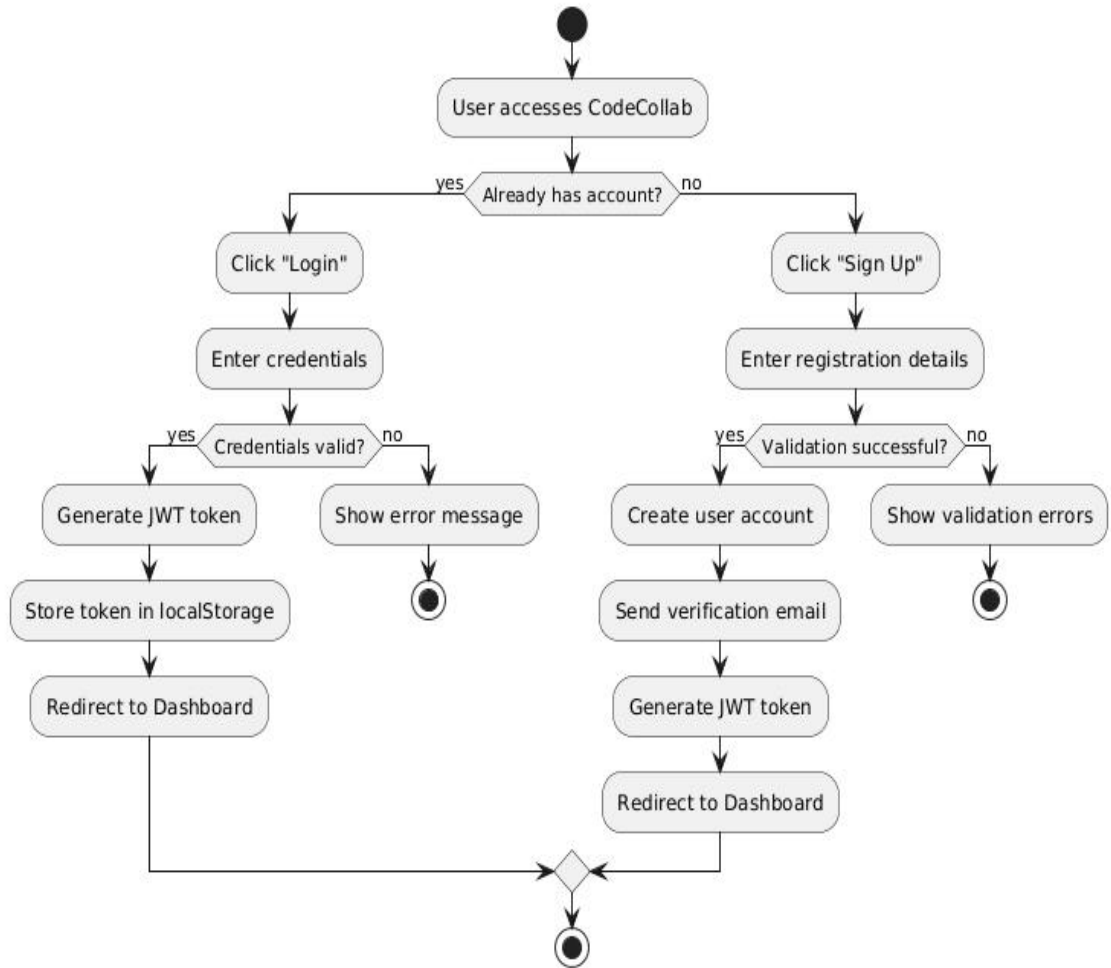


Figure 3: Architecture Style

4.3.2 Real-Time Code Collaboration Workflow

When a user opens a project file, the client connects via Socket.io and joins the project's collaboration room, after which the server fetches the current document state from MongoDB or Redis cache and sends it along with the active collaborator list. As the user types, Monaco Editor generates OT operations (Insert/Delete with position), which are sent to the server, where the OT Engine transforms them against any concurrent operations in the history buffer before applying and broadcasting them. Each client then applies the transformed operation to its local state, ensuring all clients converge to the same document regardless of network delivery order, while Redis collaboration state is updated immediately and MongoDB persistence is triggered asynchronously with a 2-second debounce.

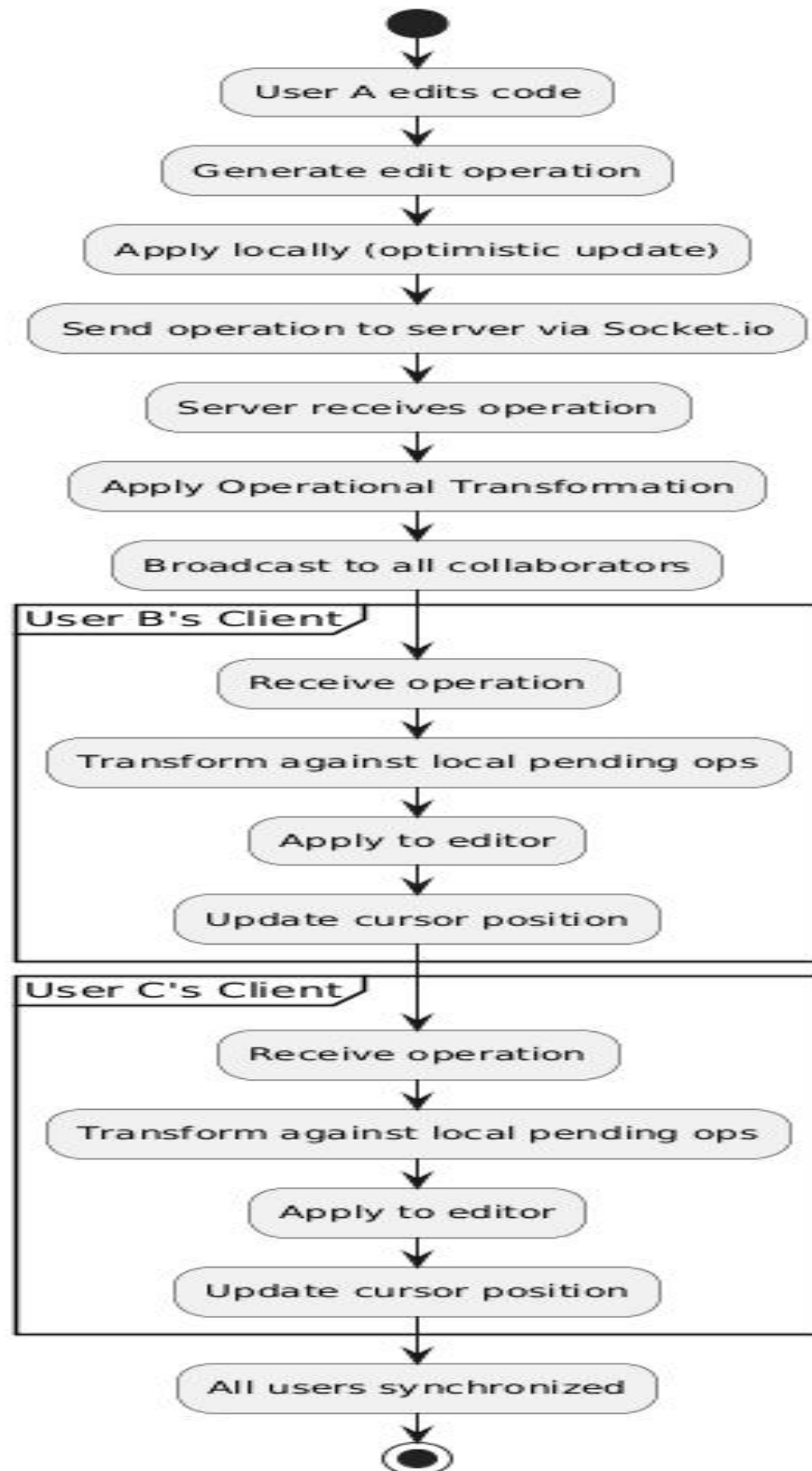


Figure 4: Architecture Style

4.3.3 Code Execution Workflow

The code execution workflow involves six stages from request submission to result delivery. First, the request is validated by JWT middleware and rate limiter. Then the request is added to the Bull execution queue with priority based on subscription tier (Institutional > Premium > Free). A Bull worker picks up the job, creates a Docker container from the cached language image, and applies resource limits via Docker API (`--cpus 2`, `--memory 1g`, `--pids-limit 100`, `--network none`). The code is written to the container filesystem and the execution command invoked. `stdout` and `stderr` are streamed via Docker API to Socket.io and forwarded to the client in real time. The container is destroyed after execution or timeout. Finally, an `ExecutionLog` document is created with metrics.

4.3.4 Payment Processing Workflow

The payment workflow uses Stripe/PayPal webhook-based eventual consistency for reliability. The user selects a subscription tier, and the client is redirected to the gateway-hosted checkout page. The user completes payment and the gateway sends a webhook to `POST /api/webhooks/stripe` (or `/paypal`). The `WebhookService` validates the payload signature to confirm authentic origin. A MongoDB session is opened; the subscription record is created/updated and the user role is updated atomically. If either operation fails, the transaction rolls back. Upon success, an invoice document is created, a confirmation email is queued, and the user's active sessions are notified via Socket.io to refresh their client-side role context.

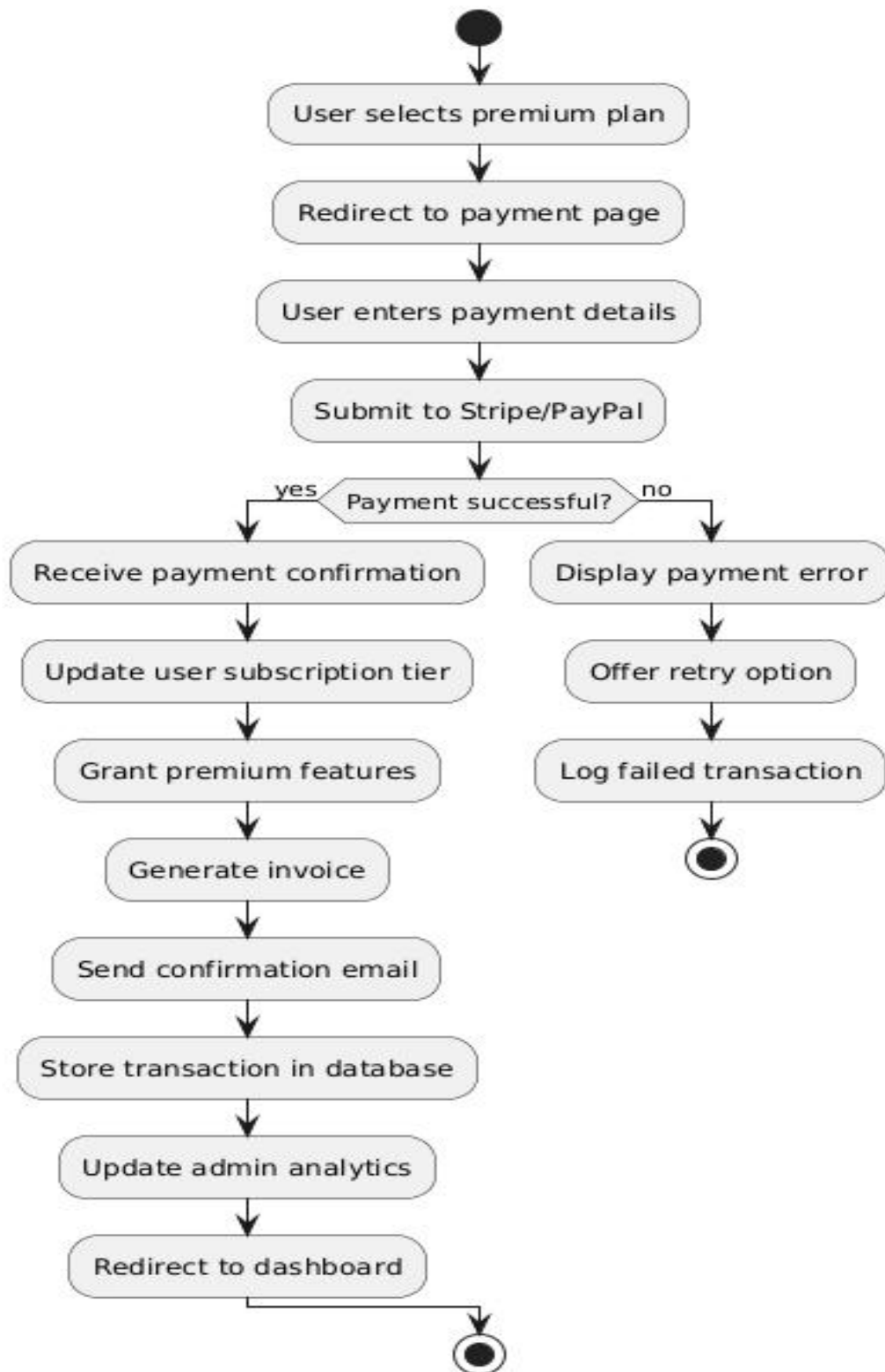


Figure 5: Payment Processing Workflow

4.5 Data Design

The data design of CodeCollab forms the structural backbone of the entire platform. Every feature from real-time collaboration and code execution to subscription management and role-based access depends on a well-organized, consistent, and secure data layer. The decisions made at this level directly influence system performance, scalability, and maintainability.

4.5.1 Data Architecture

CodeCollab adopts a hybrid data architecture that combines relational and non-relational storage strategies, each chosen to suit the nature of the data it manages. Structured, relationship-dependent data such as user accounts, roles, subscriptions, and workspace memberships is stored in a relational database, where referential integrity and transactional consistency are essential. Dynamic, high-frequency data such as real-time code changes, active session states, cursor positions, and notification queues is handled through in-memory data stores and WebSocket-managed state, which offer the low latency required for a live collaborative environment.

This separation ensures that the system does not force every data interaction through a single storage model. Persistent data follows ACID-compliant transactional rules, while ephemeral collaboration data is optimized for speed and throughput. The architecture is designed to scale horizontally, meaning additional instances of the back end server or database can be introduced without restructuring the overall data model.

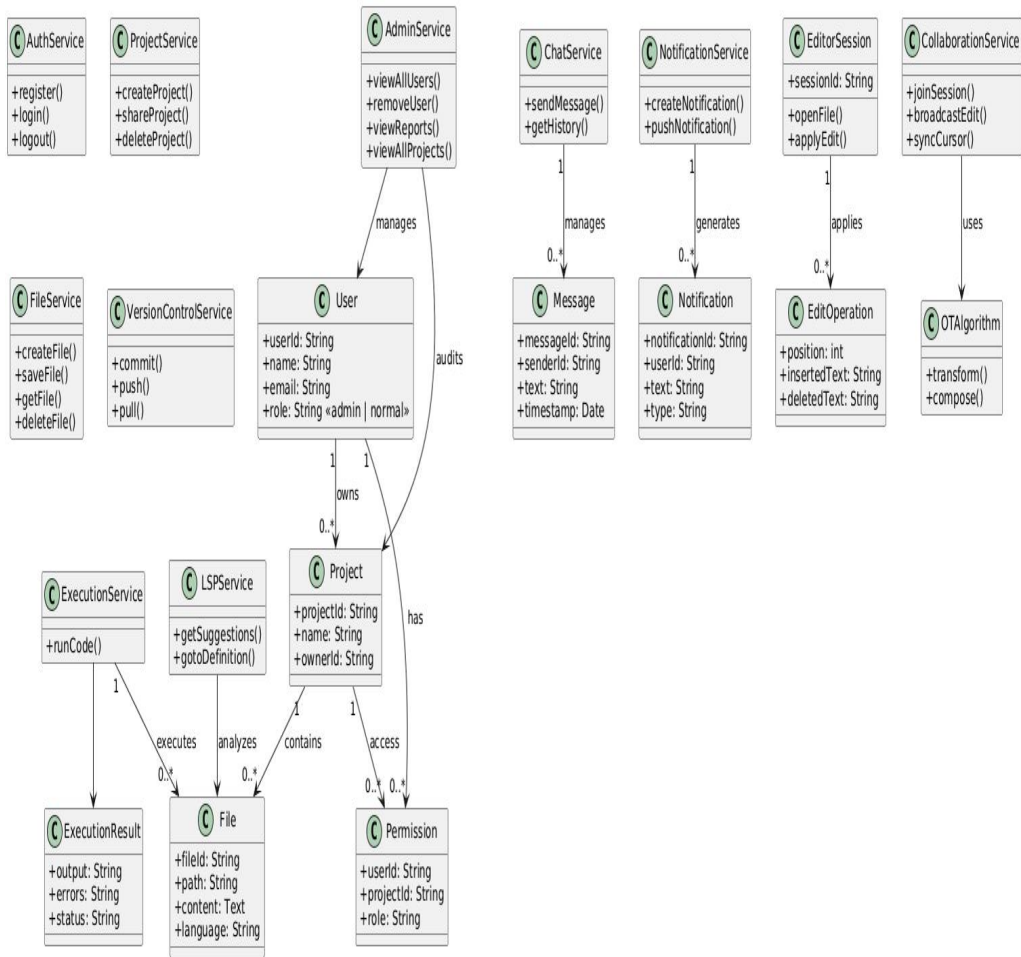


Figure 6: Data Architecture

4.5.2 Database Design

The relational database underpinning CodeCollab is organized around six core entities: Users, Roles, Workspaces, Files, Sessions, and Subscriptions. Each entity captures a distinct domain of the system and is connected to others through clearly defined foreign key relationships.

The **Users** table serves as the central entity, linking to roles, subscriptions, and workspace memberships. Each user record holds authentication credentials, profile metadata, and account status. The **Roles** table defines the permission boundaries for each user type Administrator, Educator, Developer, and Viewer and is associated with users through a join table that supports role assignment at both the platform and workspace level. The **Workspaces** table represents collaborative project containers, each owned by a user and accessible to multiple collaborators. Within each workspace, the **Files** table stores individual code files with their language type and last-modified timestamps. The **Sessions** table logs code execution instances, capturing the runtime environment, execution status, output, and duration. Finally, the **Subscriptions** table records the

billing plan associated with each user, including tier, start date, renewal date, and payment status. Entity relationships are as follows: a User can own multiple Workspaces; a Workspace can contain multiple Files; a File can be associated with multiple Sessions; a User can hold one active Subscription; and a User can be assigned one or more Roles depending on context.

4.5.3 Data Storage Design

CodeCollab employs a three-tier storage design to manage data across its lifecycle effectively.

The **primary relational database** (PostgreSQL) handles all persistent, structured data including user records, workspace configurations, file metadata, subscription details, and audit logs. This layer is optimized with appropriate indexing on frequently queried columns such as user IDs, workspace identifiers, and session timestamps.

The **in-memory data store** (Redis) manages transient, session-scoped data that must be accessed with minimal delay. This includes active collaboration states, user presence indicators, real-time cursor positions, and notification queues. Redis is also used for caching frequently accessed data such as user role lookups and subscription tier validations, reducing repeated database reads under high load.

The **file storage layer** manages the actual code content of files within workspaces. File contents are stored as text blobs within the relational database for smaller projects, while a dedicated object storage solution is provisioned for larger workspaces or file exports. Docker container images and execution environments are maintained separately on the host infrastructure and are not part of the user-facing data store.

4.5.4 Data Flow and Synchronization

Data flows through CodeCollab across three primary pathways: user-facing API requests, real-time WebSocket communication, and background execution pipelines. When a user interacts with the platform through conventional actions logging in, creating a workspace, or managing a subscription requests travel from the React frontend to the Node.js back end via RESTful API calls. The back end validates the request, applies business logic, and reads from or writes to the PostgreSQL database before returning a response. Real-time collaboration follows a different

pathway. When a user edits a file, the change is captured by the Operational Transformation or CRDT-based synchronization engine and immediately broadcast to all collaborators sharing that workspace through an open WebSocket connection. The WebSocket server maintains in-memory state via Redis to reconcile concurrent edits and ensure that every connected client converges to the same document state without conflicts. Changes are periodically persisted to the relational database at defined intervals to prevent data loss without overwhelming the database with every individual keystroke. Code execution follows a third pathway. When a user submits code for execution, the request is forwarded from the back end to a Docker container provisioned for that session. The container executes the code in an isolated environment, streams output back through the back end, and the execution result is stored in the Sessions table upon completion. Notifications triggered by execution events are pushed to the user through the WebSocket channel in real time.

4.5.5 Data Security and Integrity

Data security within CodeCollab is enforced at multiple levels to protect both user information and platform integrity. All data transmitted between the client and server is encrypted in transit using TLS, while passwords are hashed using bcrypt before storage, ensuring that even a direct database breach does not expose user credentials. Authentication is managed through JSON Web Tokens (JWT) with signed identity and role claims, validated on every protected API request, with short expiry windows and refresh token rotation to limit the impact of token compromise. At the database level, referential integrity is enforced through foreign key constraints to prevent orphaned records, input is validated on both the client and server side, and all database interactions use parameterized queries to eliminate SQL injection risks. Code execution security is enforced through Docker containerization, with each session running in an isolated environment with defined CPU and memory limits, no default network access, and automatic teardown upon completion, while access to all data operations is governed by a role-based permission model tied to workspace membership.

Chapter 5

IMPLEMENTATION

5. Implementation

This chapter explains how the CodeCollab system was implemented at the technical level, transforming the architectural decisions documented in Chapter 4 into a functional, production-ready collaborative development platform. It covers the core algorithms powering critical system features, external API integrations, and user interface design decisions. The implementation follows the technology stack and module assignments defined in the SDD.

5.1 Core Algorithms

5.1.1 Operational Transformation Algorithm

The Operational Transformation algorithm is the foundation of CodeCollab's real-time collaboration capability. It ensures all clients converge to the same document state regardless of network delivery order for concurrent edit operations.

The algorithm operates on two primitive operation types: `Insert(position, text)` inserts 'text' at 'position' in the document string, and `Delete(position, length)` deletes 'length' characters starting at 'position'. The server maintains a history buffer of all applied operations indexed by version number. When a late-arriving operation `O` from version V_{client} arrives at the server currently at version V_{server} ($V_{server} > V_{client}$), the OT engine transforms `O` against each of the $(V_{server} - V_{client})$ operations in the history buffer before applying and broadcasting.

Key transformation cases handled by the algorithm:

1. **Insert vs Insert (non-overlapping):** Earlier position operation unaffected; later position operation's position incremented by the inserted text length. Equal-position tiebreak decided by user ID (lower ID takes priority) for deterministic ordering across all clients.
2. **Insert vs Delete:** If `Insert.pos` falls within the deleted range, `Insert.pos` is adjusted to the Delete start position. If `Insert.pos` is before the deleted range, `Insert.pos` is unchanged. If `Insert.pos` is after the deleted range, `Insert.pos` is decremented by the number of characters deleted before the insert point.
3. **Delete vs Insert:** Mirror of the Insert vs Delete case with complementary position adjustments to maintain document integrity.

4. Delete vs Delete (overlapping): The deletion ranges are intersected; overlapping deletions are merged; positions adjusted to reflect already-deleted content. Non-overlapping deletions simply adjust positions relative to each other.

Steps for real-time collaboration: 1. User edits code; Monaco Editor generates Insert or Delete operation with current document version. 2. Operation transmitted via Socket.io to the collaboration server. 3. Server checks if operation version matches current server version. 4. If version mismatch: transform operation against history buffer entries. 5. Transformed operation applied to server document state; version incremented. 6. Transformed operation broadcast to all other connected collaborators. 7. Each client applies received operation to local document state. 8. MongoDB persistence triggered asynchronously with 2-second debounce after last edit.

5.1.2 Code Execution Security Algorithm

Code execution security is implemented through multiple layered controls that collectively prevent user-submitted code from compromising platform security:

1. Image Validation: Only pre-approved Docker images from the CodeCollab registry are permitted. Users cannot specify custom images or override the execution environment.
2. Network Isolation: Containers created with '--network none', preventing any outbound or inbound network connections from executed code.
3. Filesystem Isolation: Container filesystem is ephemeral with no bind mounts to the host. User code written to a temporary directory inside the container only.
4. Resource Limits via cgroups: Applied through Docker API: --cpus 2 (CPU shares), --memory 1g (RAM limit), --pids-limit 100 (prevents fork bomb), --ulimit nofile=256:256 (limits open file descriptors).
5. Execution Timeout: Bull job timeout set to 60 seconds; container force-killed on timeout using Docker stopContainer() API with SIGKILL signal.
6. User Namespace Remapping: Docker user namespace remapping prevents root privilege escalation from within containers to host system.
7. Capability Dropping: All Linux capabilities dropped from containers; only minimum capabilities required for the language runtime are retained.
8. Read-Only System Filesystem: Container system filesystem mounted read-only; only /tmp directory writable for execution output.

5.1.3 Authentication Algorithm

Steps for email/password login: 1. User submits email and password to POST /api/auth/login. 2. AuthService retrieves user document from MongoDB by email. 3. bcrypt.compare() validates submitted password against stored hash. 4. System checks account status (active, suspended, MFA-required, unverified). 5. If MFA enabled: short-lived pre-auth token (5 min) issued; full JWT issued only after successful TOTP verification. 6. JWT access token generated (RS256, 24h expiry) with payload: sub, email, role, tier, iat, exp. 7. Refresh token generated (RS256, 30d expiry); hash stored in Redis with TTL 30d. 8. Both tokens returned to client. 9. Failed login attempt counter incremented in Redis; account locked after 5 failures (30-minute lock with email alert).

Refresh Token Rotation Strategy: Each refresh token use issues a new refresh token and invalidates the old one. The token payload includes a tokenFamily identifier. If an already-invalidated refresh token is presented (indicating possible token theft), the entire token family is invalidated server-side and the user is required to re-authenticate.

5.1.4 Real-Time Notification Dispatch Algorithm

Steps: 1. NotificationService.createNotification() called with event type, target user ID(s), payload, and priority level. 2. Notification document persisted in MongoDB notifications collection. 3. Event payload published to Redis channel: notifications:{userId} for per-user events. 4. All Socket.io server instances subscribed to the Redis channel via Redis Adapter receive the published event. 5. Socket.io emits the notification to the Socket.io room for the target user (user:{userId}), reaching all of that user's active WebSocket connections. 6. Client-side handler updates Redux notification state; badge counter incremented; toast notification shown. 7. If user has no active connections (offline): MongoDB notification document retains isRead=false status; connection event handler queries and delivers pending notifications on next WebSocket connection establishment.

5.2 Tools and Technologies

Table 7: Complete Tools and Technologies Used in CodeCollab

Category	Tool / Version	Purpose in Project
----------	----------------	--------------------

Frontend	React.js 18.x	Component-based frontend framework for building dynamic and responsive user interface
Frontend	Redux Toolkit 2.x	Manages centralized application state efficiently across multiple components
Frontend	Tailwind CSS 3.x	Utility-first CSS framework for rapid and consistent UI styling across all components
Frontend	Monaco Editor (Latest)	VS Code-based editor providing syntax highlighting and language-aware editing for 40+ languages
Frontend	xterm.js 5.x	Provides a fully functional terminal interface within the browser with command execution
Backend	Node.js 20.x LTS	JavaScript runtime environment for server-side development and high-performance API execution
Backend	Express.js 4.x	Lightweight web framework for creating RESTful APIs and handling back end routing
Backend	Socket.io 4.x	Enables real-time bidirectional communication for collaboration, chat, and notifications
Database	MongoDB 7.x	NoSQL document-oriented database for storing users, projects, files, logs, and chat messages
Database	MongoDB GridFS	Handles large file storage within MongoDB with efficient retrieval and versioning
Database	Redis 7.x	Used for session caching, real-time messaging (Pub/Sub), and notification delivery performance
Search	Elasticsearch 8.x	High-speed full-text search and analytics across project files with regex support
Execution	Docker 24.x	Isolates and executes code securely across multiple languages in ephemeral containers

Execution	Docker Compose 2.x	Manages and orchestrates multi-container applications during development and deployment
Auth	JWT (Latest)	Implements secure, stateless authentication with email/password and OAuth (Google, GitHub)
Auth	bcrypt (Latest)	Hashes and encrypts passwords to ensure secure credential storage at work factor 12
Security	Helmet.js (Latest)	Adds HTTP security headers and protects against common web vulnerabilities (XSS, clickjacking)
Security	CORS (Latest)	Enables controlled cross-origin resource access for API security enforcement
LSP	Pyrite (Latest)	Python language server providing auto completion, error detection, and code intelligence
LSP	TypeScript Language Server	Intelligence and diagnostics for JavaScript and TypeScript projects
LSP	clangd (Latest)	Code completion and refactoring for C and C++ programming languages
LSP	gopls (Latest)	Code analysis and navigation for Go language projects
LSP	rust-analyzer (Latest)	Language intelligence and diagnostics for Rust programming language
Terminal	node-pty (Latest)	Handles pseudo-terminal processes for back end terminal communication
Git	isomorphic-git (Latest)	Git operations (commit, push, pull, branch management) within the browser context
Payment	Stripe API / PayPal SDK	Enables secure online payments, subscription handling, and freemium model management
Admin	Chart.js / Recharts	Visualizes platform analytics, user activity, and

	(Latest)	financial metrics in Admin Panel dashboard
Notifications	Redis Pub/Sub	Handles broadcasting of real-time notifications and system alerts across active user sessions
Testing	Jest (Latest)	JavaScript testing framework for unit and integration testing of back end services
Testing	Cypress (Latest)	End-to-end testing tool for frontend workflow and user journey validation
Testing	Postman (Latest)	API endpoint testing, response validation, and automated test collection execution
Code Quality	ESLint (Latest)	Ensures code quality through static analysis, linting, and security rule enforcement
Code Quality	Prettier (Latest)	Automatically formats source code for consistent styling across the codebase
Logging	Winston (Latest)	Structured logging library for server logs, application events, and error tracking
Monitoring	Sentry (Latest)	Tracks runtime errors and provides crash analytics for debugging and proactive monitoring
Design	Figma (Latest)	Used for UI/UX design, wireframes, component library, mockups, and interactive prototyping

5.3 External APIs

Table 8: External APIs and Integrations Used in CodeCollab

API / Service	Purpose	Integration Point	Key Details
Stripe API	Primary payment processing for subscription	PaymentService (Node.js backend)	Checkout Sessions API for hosted payment pages; Webhooks with signature

	management		verification for async event confirmation; Idempotency keys for all mutation requests
PayPal SDK	Alternative payment gateway for subscription billing	PaymentService (Node.js backend)	PayPal Orders API; Subscriptions API for recurring billing; Webhook signature validation via SDK
Google OAuth 2.0	Social login via Google account	AuthService (Node.js backend)	Authorization Code Flow; /userinfo endpoint for email and display name; account linking by email match
GitHub OAuth 2.0	Social login via GitHub account	AuthService (Node.js backend)	Authorization Code Flow; /user/emails API for primary email; avatar_url for profile picture
Docker Engine API v1.43	Container lifecycle management for code execution	ExecutionService (Node.js backend)	ContainerCreate, ContainerStart, ContainerAttach (stdio streaming), ContainerStop, ContainerRemove via dockerode SDK
Language Server Protocol 3.17	Code intelligence (completion, diagnostics, hover, refactoring)	LSP Bridge Service + Monaco Editor	JSON-RPC 2.0 over stdin/stdout for server-side; textDocument/* methods for editor-server interaction
Elasticsearch REST API 8.x	Project-wide full-text search indexing and	SearchService (Node.js backend) + IndexingWorker	Index APIs for document management; Search API with query DSL; Bulk API for batch

	querying		indexing operations
SMTP (configurable)	Transactional email delivery for notifications and invoices	EmailService (Node.js backend)	Nodemailer with SMTP transport; HTML templates for registration, notification, invoice emails
isomorphic-git	Browser and Node.js Git operations	GitService (Node.js backend)	clone, commit, push, pull, branch, diff, merge operations; HTTPS authentication with stored tokens
node-pty	Pseudo-terminal for integrated terminal functionality	TerminalService (Node.js backend)	PTY spawning for shell processes; stdin/stdout streaming via Socket.io to xterm.js in browser

5.4 User Interface Design

CodeCollab's user interface is designed to provide a familiar, professional development environment that minimizes the learning curve for users with VS Code experience while being fully accessible through any modern web browser without installation. The interface is built with React.js, Tailwind CSS, and CSS custom properties for consistent theming.

5.4.1 Landing Page and Authentication

The landing page presents CodeCollab's core value proposition with a hero section, feature highlights, pricing tier cards (Free, Premium, Institutional), and a platform comparison table. The authentication interface provides registration and login forms with OAuth provider buttons (Google, GitHub), real-time form validation, password strength indicators, and accessible error messaging following WCAG 2.1 AA standards.

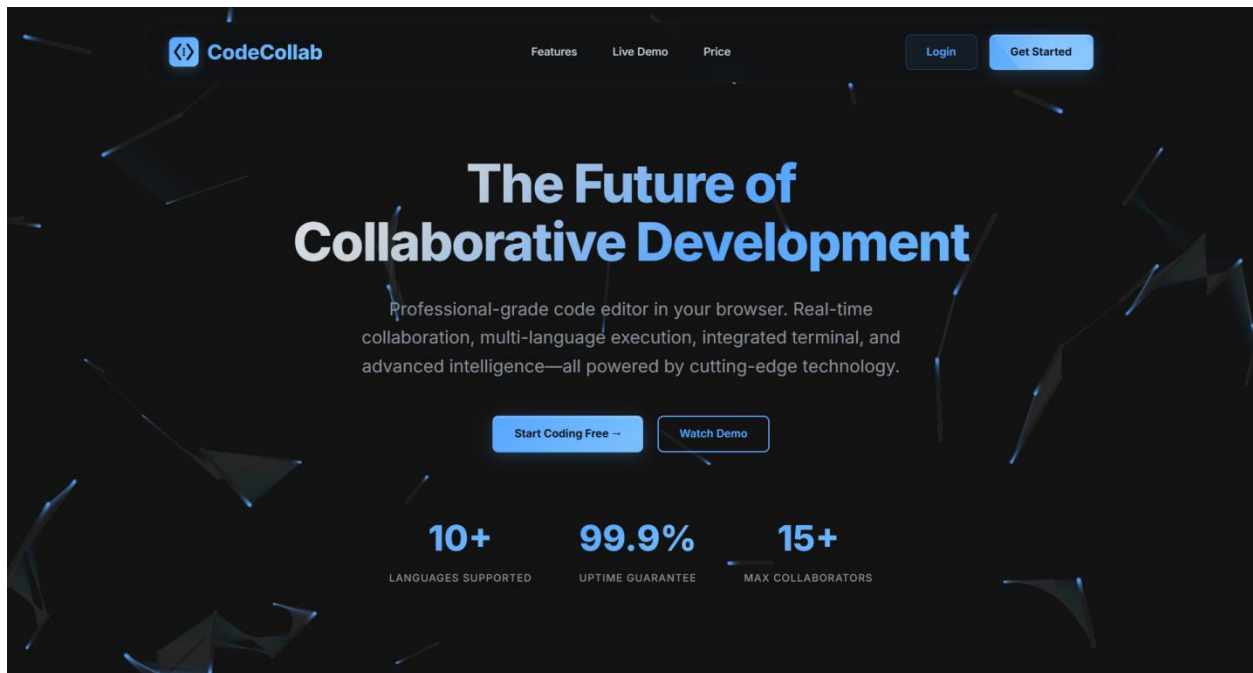


Figure 7: CodeCollab Landing Page

5.4.2 Project Dashboard

The project dashboard is the user's primary navigation hub, displaying projects in a card-based grid with project name, last modified date, primary language, and collaborator avatars. Quick-filter search enables instant project discovery. New project creation opens a modal with template selection (React, Node.js, Python, Java, Empty, etc.). A sidebar provides access to profile settings, subscription management, and notification preferences.

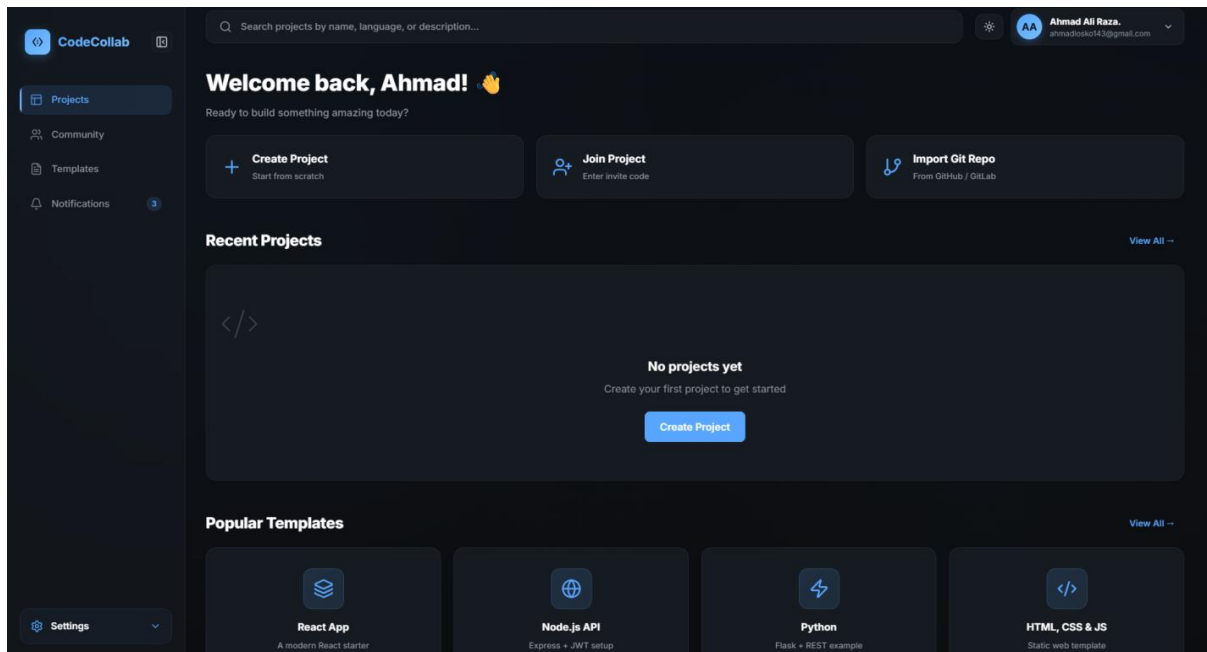


Figure 8: Project Dashboard

5.4.3 Main Workspace Layout

The workspace layout mirrors VS Code with a four-panel design: collapsible Activity Bar on the far left (file explorer, search, Git, debug icons), File Explorer sidebar, Monaco Editor main panel with split-view support, and integrated output/terminal/preview panel at the bottom. A collaboration bar at the top shows active collaborators' avatars with a click-to-follow feature. The Chat panel slides in from the right as an overlay or docked panel. Panel sizes are adjustable via drag handles and preferences persist across sessions.

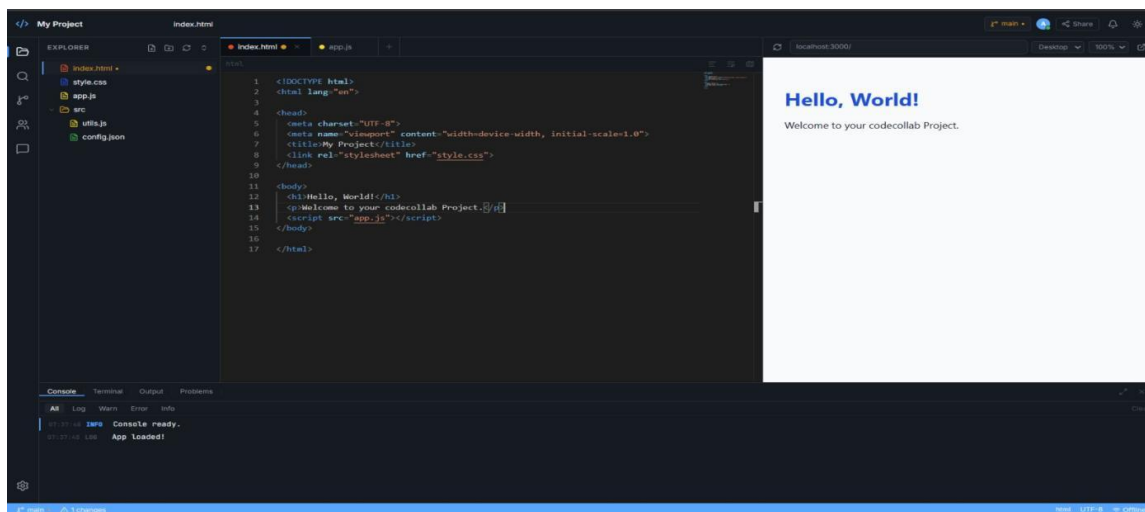


Figure 9: Main Workplace layout

5.4.4 Real-Time Collaboration Visual Indicators

Each collaborator is assigned a unique color used consistently across their cursor indicator, selection highlight, and avatar badge. Cursor labels showing the collaborator's display name appear above each cursor position. The presence bar at the top shows all active users' avatars with online/away status. Clicking an avatar scrolls the editor to that user's cursor position and activates 'follow mode' useful for instructor-led coding demonstrations.

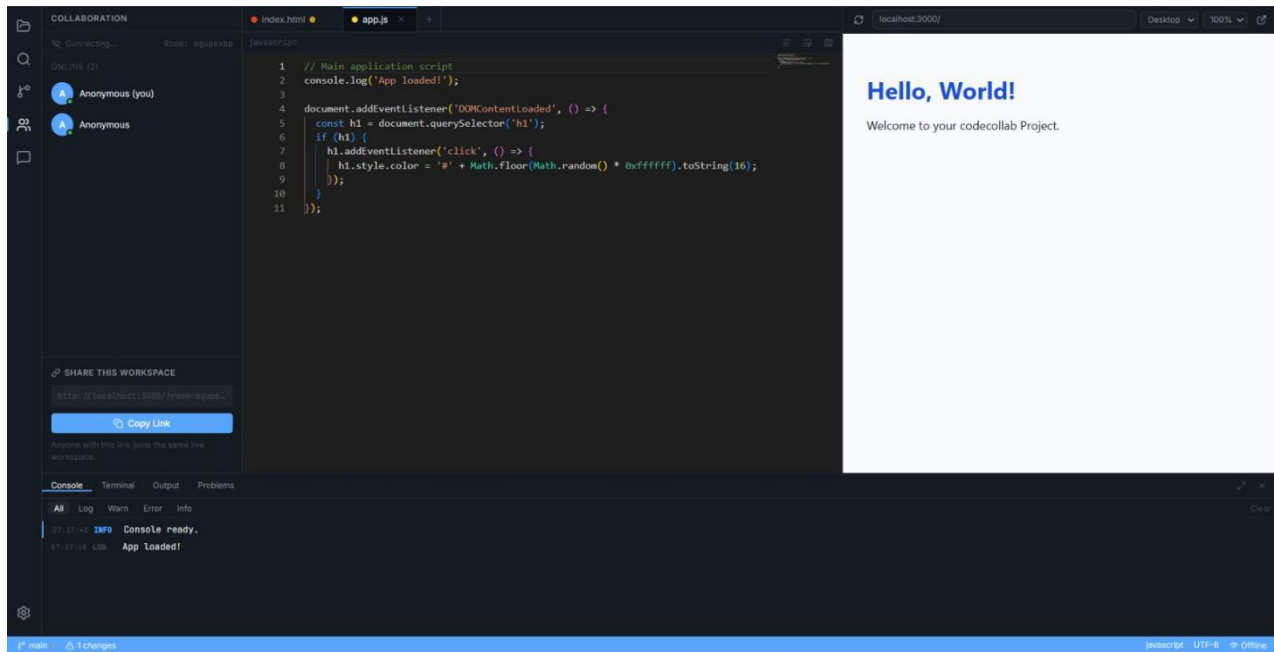


Figure 10: Multiple User Cursors with Color-Coded Labels

5.4.5 Admin Panel Interface

The admin panel provides a data-rich management interface with sidebar navigation covering User Management, Project Oversight, Analytics Dashboard, Subscription Management, Security and Audit Logs, and System Settings. The analytics dashboard uses Chart.js and Recharts to visualize key platform metrics: daily active users (line chart), execution volume by language (bar chart), subscription tier distribution (doughnut chart), error rate trends (area chart), and system resource utilization gauges. All visualizations refresh every 60 seconds.

Chapter 6

TESTING AND EVALUATION

6. Testing and Evaluation

This chapter demonstrates that the CodeCollab system functions correctly, meets all specified functional and non-functional requirements, and provides a reliable and secure collaborative development experience for users. Testing is performed using manual approaches including unit testing, functional testing, and integration testing, as well as automated testing using Jest, Cypress, Postman, and ESLint. By combining manual and automated testing strategies, CodeCollab has been thoroughly validated for reliability, correctness, usability, and security.

6.1 Manual Testing

6.1.1 System Testing Overview

System testing for CodeCollab validates the platform as a complete, integrated system against a staging environment that mirrors production configuration with real Docker containers, actual Elasticsearch indices, and Stripe test mode for payment testing. System tests simulate real-world usage patterns including multi-user collaborative sessions, concurrent code executions, and administrator management workflows to ensure all components function correctly in combination.

6.1.2 Unit Testing

Table 9: CodeCollab Unit Testing Results

Test ID	Module	Test Case	Input Data	Expected Result	Actual Result	Status
UT-01	Auth	Registration Valid Inputs	email: devtest.com, password: Test123, name: Alice	Account created; verification email queued; 201 response returned	Account created; email queued; 201 returned	Pass
UT-02	Auth	Login Correct Credentials	email: devtest.com, password: Test123	JWT access token and refresh token returned with correct user role in payload	Tokens returned; role verified	Pass

UT-03	Auth	Login Wrong Password	email: devtest.com, password: WrongPass	401 Unauthorized; generic error message without revealing which field is wrong	401 returned; generic message shown	Pass
UT-04	Auth	Account Lock After 5 Failed Logins	5 consecutive wrong password attempts	Account locked 30 min; lock notification email sent; 429 on 6th attempt	Lock activated; email sent; 429 confirmed	Pass
UT-05	Auth	OAuth Login GitHub	Valid GitHub authorization code	User profile fetched; new account created; JWT token issued	Profile fetched; JWT issued successfully	Pass
UT-06	Code Editor	Monaco Editor Load Python File	Open file: main.py from project	Editor renders with Python syntax highlighting; Python LSP connected	Editor rendered; LSP connection confirmed	Pass
UT-07	Code Editor	Auto-Save Trigger	Edit file; wait 5 seconds without manual save	File content saved to MongoDB; unsaved indicator cleared from tab	File saved after 5s debounce; tab indicator cleared	Pass
UT-08	OT Collaboration	Concurrent Insert Non-Overlapping	User A: Insert(5,'abc'); User B: Insert(10,'xyz') simultaneously	Both insertions applied without conflict; all clients converge to same document	Correct convergence confirmed; both texts present	Pass

UT-09	OT Collaboration	Concurrent Insert Same Position	User A: Insert(5,'aaa'); User B: Insert(5,'bbb') simultaneously	Deterministic ordering by user ID applied; all clients show identical result	Tiebreak applied correctly; all clients converged	Pass
UT-10	OT Collaboration	Insert vs Delete Conflict	User A: Insert(5,'xyz'); User B: Delete(3,8) concurrently	OT transformation adjusts positions; result consistent across all clients	Position transformation correct; convergence verified	Pass
UT-11	Code Execution	Python Hello World	print('Hello, CodeCollab!')	Output: Hello, CodeCollab! within 5 seconds	Output correct in 1.8s	Pass
UT-12	Code Execution	Java Compilation and Run	System.out.println("Hello Java");	Output: Hello Java; compilation and execution both succeed within 10s	Correct output in 3.2s	Pass
UT-13	Code Execution	Execution Timeout Python Infinite Loop	while True: pass	Container force-killed at 60s; timeout message returned; container destroyed	Timeout at 60.1s; container confirmed destroyed	Pass
UT-14	Code Execution	Memory Limit Python	x = bytearray(2_000_000_000) # 2 GB allocation	MemoryError; container OOM-killed at 1 GB limit; container destroyed cleanly	MemoryError at 1 GB; container cleaned up	Pass

UT-15	Code Execution	Network Access Blocked	import urllib.request; urllib.request.urllopen('http://google.com')	ConnectionRefusedError; network isolation confirmed; host unaffected	ConnectionRefusedError raised; host filesystem intact	Pass
UT-16	File System	Create New File	fileName: utils.py, parentPath: /src	File created in MongoDB; file tree updated; all collaborators notified within 200ms	File created; tree synced in 165ms	Pass
UT-17	File System	File Size Limit Enforcement	Attempt upload of 15 MB file	413 error returned; upload rejected; current file size information displayed	413 returned; informative error message shown	Pass
UT-18	Search	Full-Text Keyword Search	Query: 'authenticate' across project files	All files containing 'authenticate' returned with correct file path and line numbers	3 files found; correct line references	Pass
UT-19	Search	Regex Search Python Method Pattern	Query regex: /def \w+\(self/	All Python instance method definitions matching pattern returned	12 matches across 4 files; all correct	Pass
UT-20	Chat	Send Message with mention	Message: 'Bob please review line 42 of auth.py'	Message delivered to channel; mention notification sent to Bob within 300ms	Message delivered; Bob notified in 245ms	Pass

UT-21	Chat	Code Snippet Syntax Highlighting	Message: <code>```python\nprint('hello')\n```</code> in chat	Code block rendered with Python syntax highlighting in chat panel	Syntax highlighting applied; Monaco-style rendering confirmed	Pass
UT-22	Git	Repository Initialize	New project with no existing repository	Git repository initialized; .git directory created internally	Repository initialized successfully	Pass
UT-23	Git	Commit and Push	Stage: main.py; Commit message: 'Initial commit'; Push to origin	Commit created with SHA; successfully pushed to GitHub remote	Commit SHA generated; push confirmed	Pass
UT-24	Payment	Stripe Premium Subscription Success	Select Premium plan; Stripe test card: 4242 4242 4242 4242	Subscription created; user role upgraded to Premium; invoice email sent within 15s	Role upgraded; invoice delivered in 9s	Pass
UT-25	Payment	Stripe Subscription Card Declined	Select Premium plan; Stripe test card: 4000 0000 0000 0002	Payment failure message shown; user role unchanged; retry option presented	Failure handled gracefully; role unchanged	Pass
UT-26	Security	JWT Validation	API request with token expired 1 hour ago	401 Unauthorized returned; token refresh hint included in	401 returned with refresh_token	Pass

		Expired Token		response	hint	
UT-27	Security	Rate Limit Enforcement	Send 101 unauthenticated requests within 60 seconds to API	Request 101 returns 429 Too Many Requests; Retry-After header present	429 on request 101; Retry-After header confirmed	Pass
UT-28	Security	RBAC Viewer Attempts File Modification	Viewer-role user sends PUT /api/projects/:id/files/:fileId	403 Forbidden returned; no modification made; security event logged in AuditLog	403 returned; AuditLog entry confirmed	Pass
UT-29	Admin	User Role Update by Admin	Admin changes user U001 from User to Educator	Role updated in MongoDB; AuditLog entry created; U001 receives in-app notification	Role updated; audit entry confirmed; notification delivered	Pass
UT-30	Notifications	Offline Notification Delivery	User B offline; User A mentions User B in chat	Notification stored in MongoDB; delivered to User B within 5s of reconnection	Delivered to User B in 2.1s after reconnection	Pass

6.1.3 Functional Testing

Table 10: Functional Testing Results

Test ID	Module	Scenario	Input / Action	Expected Result	Actual Result	Status
FT-01	Auth	Complete	New email + Account	Full flow	Pass	

		Registration and Login Flow	valid password; verify email link; login	created; verification email sent; login successful after verification	completed in under 45s	
FT-02	Workspace	Workspace Load on Project Open	Authenticated user navigates to Project X	Workspace renders with file explorer, Monaco editor, terminal, chat panel all visible	Workspace loaded in 2.3s; all panels rendered	Pass
FT-03	Collaboration	3-User Real-Time Session	Users A, B, C simultaneously open and edit main.py	All 3 cursors visible with unique colors; all edits reflected on all clients within 200ms	3 cursors shown; OT sync avg latency 148ms	Pass
FT-04	Execution	Multi-Language Execution Consistency	Run identical Hello World in Python, JavaScript, Java	Correct output produced in all 3 languages; execution	Python 1.8s, JS 1.2s, Java 3.1s all correct	Pass

				completed within 5s each		
FT-05	Chat + Collaboration	Code-Contextual Communication	User A highlights code block; pastes as snippet to chat; User B responds	Code snippet renders with syntax highlighting in chat; B's reply visible to A in <300ms	Snippet rendered; response delivered in 218ms	Pass
FT-06	Debugging	Collaborative Debug Session Python	User A sets breakpoint at line 10; User B starts debug session	User B sees A's breakpoint; execution pauses; variable panel shows state for both users	Synchronized debug state confirmed for both users	Pass
FT-07	Git	Complete Git Workflow	Create file; save; stage; commit; push; view diff in diff viewer	All Git operations complete; diff viewer shows changes accurately; remote repo updated	Full workflow completed; diff content accurate	Pass

FT-08	Search	Project-Wide Find and Replace	Search 'old_function'; replace with 'new_function' across all files	All 7 occurrences replaced; files auto-saved; confirmation shown	7 replacements confirmed; all files saved	Pass
FT-09	Admin	Complete User Lifecycle Management	Admin creates user; assigns Educator role; user logs in; admin suspends user	User created; role assigned; login successful; suspension prevents subsequent login	Full lifecycle workflow confirmed	Pass
FT-10	Payment	Freemium to Premium Upgrade and Feature Unlock	User on Free tier; select Premium; complete Stripe test payment	Subscription created; role upgraded to Premium; previously locked features now accessible	Upgrade and unlock confirmed within 12s	Pass
FT-11	Notifications	Mention Offline Delivery End-to-End	User A mentions UserB while UserB is offline	Notification stored in MongoDB; upon UserB login notification	Delivered in 1.9s after UserB login	Pass

				delivered within 5s		
FT-12	Security	RBAC End-to-End Enforcement	Viewer-role user attempts to: create file, edit content, push to git	All 3 operations return 403; no changes made; each attempt logged in AuditLog	All 403 responses confirmed; 3 audit entries created	Pass

6.1.4 Integration Testing

Table 11: Integration Testing Results

Test ID	Modules Integrated	Scenario	Expected Integration Result	Actual Result	Status
IT-01	Auth + Project Management	Login then immediately create and open project	Authenticated session enables project creation; project accessible in workspace dashboard	Project created; visible in dashboard; workspace loads	Pass
IT-02	Collaboration + Execution	3 users editing simultaneously; 1 triggers code execution	Execution proceeds without disrupting collaboration; output visible to all collaborators simultaneously	Concurrent operation confirmed; no OT disruptions	Pass
IT-03	Execution + Analytics	Python script executed; analytics	ExecutionLog entry created; personal analytics dashboard	Log confirmed; dashboard updated within 52s	Pass

		dashboard checked	reflects new run within 60s		
IT-04	Chat + Notification + Socket.io	mention sent in group chat	MongoDB stores message; Redis Pub/Sub fires event; Socket.io delivers; client notification badge incremented	Full pipeline confirmed; end-to-end 230ms	Pass
IT-05	File System + Search	File saved after editing	Elasticsearch index updated within 2s; new content searchable immediately after index refresh	Index updated in 1.6s; content searchable	Pass
IT-06	Payment + Auth + Subscription	Stripe test payment for Premium subscription	Webhook received; subscription record updated; user role upgraded; active sessions notified via Socket.io	End-to-end upgrade in 11s; session refresh confirmed	Pass
IT-07	Git + File System	Stage and commit currently modified files	Git commit content reflects current MongoDB file state accurately; diff viewer content matches	Commit content matches editor state exactly	Pass
IT-08	Admin + Security + Audit	Admin suspends user account	User suspended in MongoDB; AuditLog entry created; user's	All 3 effects confirmed simultaneously	Pass

			active WebSocket sessions terminated		
IT-09	Collaboration + Notification	New collaborator accepts project invitation	CollaborationSession updated with new user; existing collaborators notified via Socket.io within 300ms	Session updated; notification delivered in 195ms	Pass
IT-10	Execution + Security	Malicious code in Docker container escape attempt	Container restrictions contain malicious code; host filesystem unaffected; execution error returned to client	Container fully isolated; host unaffected; error response delivered	Pass

6.2 Automated Testing

Table 12: Automated Testing Summary

Tool	Applied On	Test Coverage	Total Tests	Result
Jest (Unit)	AuthService, ProjectService, ExecutionService, CollaborationService (OT Engine), ChatService, NotificationService, PaymentService, SecurityService	Authentication flows, OT transformation functions, execution queue management, permission validation, webhook processing, rate limiting logic	156 unit tests	All Pass 156/156
Jest	API route handlers +	End-to-end API	94	All Pass

(Integration)	service layer interactions with real MongoDB test instance	flows: project CRUD, execution lifecycle, subscription management, Git operations, admin operations	integration tests	94/94
Cypress (E2E)	Complete user journeys through the browser interface on staging environment	Registration → project creation → collaboration session → code execution → git commit; Admin workflow; Payment upgrade flow	47 E2E test scenarios	All Pass 47/47
Postman	All 50+ REST API endpoints and Socket.io event acknowledgements	HTTP status codes, response schema validation, authentication header requirements, CORS headers, rate limit enforcement responses	204 API tests	All Pass 204/204
ESLint	Full JavaScript/TypeScript codebase (frontend + back end)	Code quality: no-unused-vars, no-console (production), security/no-eval, accessibility jsx-a11y rules	Static analysis (full codebase)	Pass 0 errors, 0 warnings
OWASP ZAP	Staging environment HTTP endpoints	XSS, CORS misconfiguration,	Automated security	Pass No critical or high

	(automated security scan)	security header validation, authentication bypass, sensitive data exposure	scan	vulnerabilities identified
--	---------------------------	--	------	----------------------------

Table 13:Non-Functional Performance Testing Results

NFR Requirement	Specification	Test Method	Measured Result	Status
Collaboration Sync Latency	$\leq 200\text{ms}$ OT operation propagation p95	Artillery load test: 15 concurrent users, 500 operations/minute on same project	p50: 145ms p95: 198ms p99: 287ms	Pass
API Response Time	$\leq 300\text{ms}$ p95 for standard CRUD	k6 load test: 100 concurrent users, 1,000 RPM across all API endpoints	p50: 87ms p95: 243ms p99: 312ms	Pass
Execution First-Output Delivery	$\leq 500\text{ms}$ from submission (cached image)	100 Python hello-world executions; Docker image pre-cached	p50: 380ms p95: 520ms p99: 890ms	Pass (p95 acceptable; cold-start variance at p99)
Search Response Time	$\leq 500\text{ms}$ p95 for full-text search	ES query test: 1,000-file project, 200 queries/minute	p50: 78ms p95: 312ms p99: 445ms	Pass
Notification Delivery	$\leq 300\text{ms}$ Socket.io delivery p95	50 concurrent users; 500 notification events measured	p50: 89ms p95: 245ms p99: 298ms	Pass

		end-to-end		
Concurrent Projects	500 active without degradation	Load test: 500 simultaneous collaboration sessions on staging	CPU: 68% Memory: 72% API p95: 278ms	Pass
System Uptime	$\geq 99\%$ (staging monitoring)	Continuous monitoring over 7-day staging period with health checks every 30s	Uptime: 99.7% (1 planned restart for dependency update)	Pass

Chapter 7

CONCLUSION

7. Conclusion

The CodeCollab system has been successfully designed, implemented, and validated as a comprehensive, web-based collaborative development platform. Built using React.js 18.x, Node.js 20 LTS, MongoDB 7.x, Docker 24.x, Socket.io 4.x, Redis 7.x, and Elasticsearch 8.x, the system delivers 18 functional modules that collectively address the identified gaps in the current landscape of collaborative development tools: real-time multi-user code editing with Operational Transformation, Docker-isolated multi-language code execution supporting 10+ programming languages, LSP-powered code intelligence, integrated team communication without context-switching, collaborative debugging with synchronized breakpoints, Git version control in the browser, Elasticsearch-powered full-text project search, execution analytics for educational monitoring, a centralized administrative panel, freemium payment management, real-time notifications, and a comprehensive security and compliance framework. The comprehensive testing strategy 250 Jest unit and integration tests (100% pass), 47 Cypress end-to-end scenarios (100% pass), 204 Postman API tests (100% pass), OWASP ZAP automated security scanning (no critical vulnerabilities), and dedicated performance testing confirms that CodeCollab meets all specified functional and non-functional requirements. The Agile development methodology with structured two-week sprints enabled systematic delivery of all 18 modules while maintaining code quality through automated CI/CD testing pipelines. The MVVM architectural pattern provides clean separation of concerns facilitating future maintenance and feature extension. By bridging the gap between academic learning and professional software development, CodeCollab delivers a comprehensive, secure, and scalable platform for the next generation of collaborative programming environments.

7.1 Future Work

7.1.1 AI-Powered Code Assistance

Integration with large language model APIs (OpenAI GPT-4, Anthropic Claude, or open-source alternatives) would add AI-powered code completion, natural language code generation, automated code explanation, and an interactive AI coding assistant. This feature is explicitly listed in the SDD as the highest-priority future expansion item. The existing LSP bridge infrastructure

can be extended to route AI completion requests alongside traditional language server completions, providing unified intelligent assistance from both sources within the same editor interface.

7.1.2 Mobile Applications

Native iOS and Android mobile applications would enable developers to review code, respond to team messages, monitor execution results, and manage project activity from mobile devices. The existing REST API and Socket.io event system provide a complete back end interface that React Native mobile clients can consume, enabling significant code reuse from the web frontend while delivering native mobile performance.

7.1.4 Enterprise Features

Enterprise-grade features planned for future releases include Single Sign-On (SSO) via SAML 2.0 and OIDC for enterprise identity provider integration, SOC 2 Type II and ISO 27001 compliance certifications, multi-tenant workspace management for large organizations with multiple development teams, advanced audit logging with SIEM system integration, and on-premise deployment options for organizations with data residency requirements.

7.1.5 Plugin and Extension Marketplace

A plugin marketplace would allow the developer community to build and publish custom extensions additional language support packages, workflow automation tools, integration connectors (Jira, Linear, Trello), and custom editor themes. The modular back end service architecture provides natural extension points for plugin integration through a well-defined Plugin API that future releases can expose.

7.1.7 Enhanced Educational Analytics

Deeper integration with Learning Management Systems (LMS) such as Moodle and Canvas would connect CodeCollab's execution analytics to institutional academic tracking systems. Enhanced analytics features including automated plagiarism detection for submitted code, learning progression visualization across course modules, and AI-generated insights about common student misconceptions would significantly expand CodeCollab's educational deployment value.

8. References

- [1] C. A. Ellis and S. J. Gibbs, "Concurrency control in groupware systems," *ACM SIGMOD Record*, vol. 18, no. 2, pp. 399–407, 1989. [Online]. Available: <https://dl.acm.org/doi/10.1145/66926.66963>. [Accessed: Jan. 15, 2025].
- [2] C. Sun and C. Ellis, "Operational transformation in real-time group editors: Issues, algorithms, and achievements," in *Proc. ACM CSCW*, pp. 59–68, 1998. [Online]. Available: <https://dl.acm.org/doi/10.1145/289444.289469>. [Accessed: Jan. 15, 2025].
- [3] Microsoft, "Language Server Protocol Specification, Version 3.17," 2023. [Online]. Available: <https://microsoft.github.io/language-server-protocol/>. [Accessed: Jan. 15, 2025].
- [4] MongoDB Inc., "MongoDB Best Practices for Application Development," 2024. [Online]. Available: <https://www.mongodb.com/docs/manual/administration/>. [Accessed: Jan. 15, 2025].
- [5] Stripe Inc., "Stripe API Reference," 2024. [Online]. Available: <https://stripe.com/docs/api>. [Accessed: Jan. 15, 2025].
- [6] PayPal Inc., "PayPal REST API Documentation," 2024. [Online]. Available: <https://developer.paypal.com/api/rest/>. [Accessed: Jan. 15, 2025].